Date Published: April 01, 2026

Visit the Zebra Product Support page for the most up-to-date version of this documentation.

Mdlocr functions

Synopsis

The functions prefixed with Mdlocr make up the Deep Learning OCR module. The Deep Learning OCR module performs optical character recognition on text using deep learning based technologies. Unlike other string reading modules, the Deep Learning OCR module does not require a font definition. By default, the Deep Learning OCR module will read all text in frame. String models and constraints can be set to narrow the results.

Functions

MdlocrAddImageToDataset
MdlocrAlloc
MdlocrAllocDataset
MdlocrAllocResult
MdlocrControl
MdlocrControlConstraint
MdlocrControlStringModel
MdlocrCopy
MdlocrDefineModel
MdlocrDefineModelFromResult
MdlocrDeleteLastImageFromDataset
MdlocrDraw
MdlocrExport
MdlocrFinetune
MdlocrFree
MdlocrGetHookInfo
MdlocrGetResult
MdlocrGetStringIndex
MdlocrHookFunction
MdlocrImport
MdlocrInquire
MdlocrInquireConstraint
MdlocrInquireStringModel
MdlocrPreprocess
MdlocrRead
MdlocrRestore
MdlocrSave
MdlocrStream

MdlocrAddImageToDataset

Synopsis

Add an image and read results to a Deep Learning OCR dataset.

Syntax

```
void MdlocrAddImageToDataset(
    AIL_ID DlocrReadResultId, //in
    AIL_ID DlocrDatasetId,    //out
    AIL_INT64 ControlFlag     //in
)
```

Description

This function adds an image and selected results from a Deep Learning OCR read result buffer to a Deep Learning OCR dataset. If required, you can delete the most recently added dataset entry, using MdlocrDeleteLastImageFromDataset().

Parameters

DlocrReadResultId

Specifies the identifier of the source Deep Learning OCR read result buffer from which to get the image and results.

DlocrDatasetId

Specifies the identifier of the destination Deep Learning OCR dataset in which to add the image and results.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrAlloc

Synopsis

Allocate a Deep Learning OCR context.

Syntax

```
AIL_ID MdlocrAlloc(
    AIL_ID SysId,             //in
    AIL_INT64 ContextType,    //in
    AIL_INT64 ControlFlag,    //in
    AIL_ID *ContextDlocrIdPtr //out
)
```

Description

This function allocates a Deep Learning OCR read context, or a Deep Learning OCR fine-tuning context on the specified system. A Deep Learning OCR read context contains information required to perform an MdlocrRead() operation, which allows you to read strings of text without defining a font. A Deep Learning OCR fine-tuning context is used to perform an MdlocrFinetune() operation on a Deep Learning OCR read context with a dataset, allocated using MdlocrAllocDataset(). This allows you to train a predefined read context on images from your use-case.

By default, a Deep Learning OCR read context is set to read all text in an image within the default character height bounds. If you want to restrict the string that is found and read, add a string model; string models are optional. MdlocrDefineModelFromResult() allows you to define the string model so that attributes are automatically set from a resulting string found using MdlocrRead(). If you only want to restrict the number of characters in the string, you can use MdlocrDefineModel(). Regardless of how you add a model, you can always constrain the characters that appear at certain positions using MdlocrControlConstraint() and adjust general string model settings, such as, character height and number of characters using MdlocrControlStringModel().

A Deep Learning OCR context internally handles characters in Unicode.

When allocating the read context, you should specify whether to use an algorithm that favors robustness or speed should be used, depending on the complexity of your images and your processing power restrictions.

Global settings and string model-specific settings and positional constraints are all held in a Deep Learning OCR read context; use MdlocrControl(), MdlocrControlConstraint(), and MdlocrControlStringModel() to set these up, respectively.

When the Deep Learning OCR context is no longer required, release it using MdlocrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the Deep Learning OCR context's lifetime and you must not manually free it.

Parameters

SysId INQ

Specifies the identifier of the system on which to allocate the Deep Learning OCR context. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ContextType INQ

Specifies the type of context to allocate. Set this parameter to the value below:

To allocate a Deep Learning OCR read context, specify one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of read context to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OCR_NET1_BALANCED_V1	Specifies to allocate a Deep Learning OCR read context with a balanced deep learning model based on the NET1 architecture. The performance and accuracy of this context falls between M_OCR_NET1_FAST_V1 and M_OCR_NET1_EXTENDED_V1. This context should be used on average complexity images and when processing power restrictions are not too high.	c		e	f	g	a		i		j	k	l	m	n	b
M_OCR_NET1_EXTENDED_V1	Specifies to allocate a Deep Learning OCR read context with the most robust deep learning model based on the NET1 architecture. Maximum accuracy over M_OCR_NET1_FAST_V1 and M_OCR_NET1_BALANCED_V1 comes at the cost of speed. This context should be used on high complexity images when processing power restrictions are low.	c		e	f	g	a		i		j	k	l	m	n	b
M_OCR_NET1_FAST_V1	Specifies to allocate a Deep Learning OCR read context with the fastest deep learning model based on the NET1 architecture. Faster performance over M_OCR_NET1_EXTENDED_V1 and M_OCR_NET1_BALANCED_V1 comes at the cost of lower accuracy. This context should be used on simpler images and when processing power is limited.	c		e	f	g	a		i		j	k	l	m	n	b
M_OCR_NET2_TINY_V1	Specifies to allocate a Deep Learning OCR read context with the tiny deep learning model based on the NET2 architecture. This context can be fine-tuned, using MdlocrFinetune() with a Deep Learning OCR dataset (M_OCR_NET2_DATASET) and a Deep Learning OCR fine-tuning context (M_OCR_NET2_FINETUNE_CONTEXT).	c		e	f	g	a		i		j	k	l	m	n	b

To allocate a Deep Learning OCR fine-tuning context, specify the following value.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of fine-tuning context to allocate																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OCR_NET2_FINETUNE_CONTEXT	Specifies to allocate a Deep Learning OCR fine-tuning context. This can be used with Deep Learning OCR read contexts based on the NET2 architecture (M_OCR_NET2_TINY_V1).	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextDlocrIdPtr

Specifies the address of the variable in which to write the Deep Learning OCR context identifier or specifies the data type that the function should use to return the Deep Learning OCR context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated Deep Learning OCR context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated Deep Learning OCR context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_DLOCR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the Deep Learning OCR context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the fine-tuning identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Deep Learning OCR fine-tuning context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the read context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Deep Learning OCR read context. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the Deep Learning OCR context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_DLOCR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.

DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrAllocDataset

Synopsis

Allocate a Deep Learning OCR dataset.

Syntax

```
AIL_ID MdlocrAllocDataset(
    AIL_ID SysId,          //in
    AIL_INT64 DatasetType, //in
    AIL_INT64 ControlFlag, //in
    AIL_ID *DatasetIdPtr   //out
)
```

Description

This function allocates a Deep Learning OCR dataset on the specified system. A Deep Learning OCR dataset contains information about images and the characters represented in these images. The dataset can be exported using MdlocrExport() and retrieved using MdlocrImport(). You can add and remove dataset entries using MdlocrAddImageToDataset() and MdlocrDeleteLastImageFromDataset(), respectively.

When the Deep Learning OCR dataset is no longer required, release it using MdlocrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the Deep Learning OCR dataset's lifetime and you must not manually free it.

Parameters

SysId

Specifies the identifier of the system on which to allocate the Deep Learning OCR dataset. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

DatasetType

Specifies the type of dataset to allocate. Set this parameter to the value below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of dataset to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_OCR_NET2_DATASET	Specifies to allocate a Deep Learning OCR dataset based on the NET2 architecture. This dataset can be used with M_OCR_NET2_TINY_V1 contexts.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

DatasetIdPtr

Specifies the address of the variable in which to write the Deep Learning OCR dataset identifier or specifies the data type that the function should use to return the Deep Learning OCR dataset identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated Deep Learning OCR dataset ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b
M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated Deep Learning OCR dataset ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_DLOCR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the Deep Learning OCR dataset (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b

Address in which to write the identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Deep Learning OCR dataset. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b	

Return value

The returned value is the Deep Learning OCR dataset identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_DLOCR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrAllocResult

Synopsis

Allocate a Deep Learning OCR result buffer.

Syntax

```
AIL_ID MdlocrAllocResult(
    AIL_ID SysId,            //in
    AIL_INT64 ResultType,    //in
    AIL_INT64 ControlFlag,   //in
    AIL_ID *DlocrResultIdPtr //out
)
```

Description

This function allocates a Deep Learning OCR read result buffer, or a Deep Learning OCR fine-tuning result buffer on the specified system, to store results obtained from an MdlocrRead() operation, or an MdlocrFinetune() operation, respectively.

When the Deep Learning OCR result buffer is no longer required, release it using MdlocrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the Deep Learning OCR result buffer's lifetime and you must not manually free it.

Parameters

SysId

Specifies the identifier of the system on which to allocate the Deep Learning OCR result buffer. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result buffer to allocate.

To allocate a Deep Learning OCR read result buffer, specify one of the following values:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of read result buffer to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_OCR_NET1_READ_RESULT	Specifies to allocate a Deep Learning OCR result buffer to hold results from a read context using the NET1 architecture.	c		e	f	g	a		i		j	k	l	m	n	b
M_OCR_NET2_READ_RESULT	Specifies to allocate a Deep Learning OCR result buffer to hold results from a read context using the NET2 architecture.	c		e	f	g	a		i		j	k	l	m	n	b

To allocate a Deep Learning OCR fine-tuning result buffer, specify the following value:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of fine-tuning result buffer to allocate																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_OCR_NET2_FINETUNE_RESULT	Specifies to allocate a Deep Learning OCR result buffer to hold results from a fine-tuning context using the NET2 architecture.	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

DlocrResultIdPtr

Specifies the address of the variable in which to write the Deep Learning OCR result buffer identifier or specifies the data type that the function should use to return the Deep Learning OCR result buffer identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated Deep Learning OCR result buffer ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated Deep Learning OCR result buffer ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_DLOCR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the Deep Learning OCR result buffer (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the fine-tuning result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Deep Learning OCR fine-tuning result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the read result buffer identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Deep Learning OCR read result buffer. If allocation fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the Deep Learning OCR result buffer identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_DLOCR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.

DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrControl

Synopsis

Control a Deep Learning OCR context, result buffer, or dataset.

Syntax

```
void MdlocrControl(
    AIL_ID ContextOrResultDlocrId, //out
    AIL_INT64 ControlType,         //in
    AIL_DOUBLE ControlValue        //in
)
```

Description

This function allows you to control a Deep Learning OCR context, a Deep Learning OCR result buffer, or a Deep Learning OCR dataset. To inquire about settings in a context, result buffer, or a dataset, call MdlocrInquire(). To get results from a result buffer, call MdlocrGetResult().

To control the string model-specific settings and positional constraints held in a Deep Learning OCR read context, call MdlocrControlStringModel() and MdlocrControlConstraint(), respectively.

You must preprocess the Deep Learning OCR context after you have finished adjusting context settings and before calling MdlocrRead() or MdlocrFinetune(). To know if a context needs to be preprocessed, call MdlocrInquire() with M_PREPROCESSED.

Parameters

ContextOrResultDlocrId

Specifies the identifier of the Deep Learning OCR context, result buffer, or dataset to control. The context must have been previously allocated on the system using MdlocrAlloc(). The result buffer must have been previously allocated on the system using MdlocrAllocResult(). The dataset must have been previously allocated on the system using MdlocrAllocDataset().

ControlType

Specifies the type of control to set.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the control.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The tables below list possible values for the **ControlType** and **ControlValue** parameters.

- For controlling a Deep Learning OCR read context setting
- For controlling the predict engine
- For controlling a Deep Learning OCR read result buffer
- For controlling a Deep Learning OCR fine-tuning context
- For controlling a Deep Learning OCR fine-tuning result buffer
- For controlling a Deep Learning OCR read result buffer, fine-tuning result buffer, or dataset

The following ControlType and corresponding ControlValue parameter settings are used to control a setting of the Deep Learning OCR read context specified with the ContextOrResultDlocrId parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a Deep Learning OCR read context setting																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																	
M_ACCEPTED_CHARS	Sets the list of accepted characters. The characters in this list should be among those supported by the context type; use MdlocrInquire() with M_SUPPORTED_CHARS to establish the list of supported characters. Note that calls to **MdlocrControl**() with M_ACCEPTED_CHARS are not accumulative. To get the list of characters in a predefined list, set M_ACCEPTED_CHARS to the required predefined list, and then inquire what characters this corresponds to using MdlocrInquire() with M_ACCEPTED_CHARS. INQ																
M_DEFAULT_ACCEPTED_CHARS	Specifies the list of all letters (upper and lowercase) and digits. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_DIGITS_CHARS	Specifies the list of all numeric digits.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTERS_CHARS	Specifies the list of all upper and lowercase letters.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTERS_LOWERCASE_CHARS	Specifies the list of all lowercase letters.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTERS_UPPERCASE_CHARS	Specifies the list of all uppercase letters.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Accepted characters*") 1	Specifies the list of accepted characters.	c		e	f	g	a		i		j	k	l	m	n	b

M_CHAR_ACCEPTANCE	Sets the acceptance level for the character score. To be accepted as a read result, the character's score must be greater than or equal to this value. The character's score quantifies the confidence that the Deep Learning OCR classifier has correctly predicted the character. Character scores can be retrieved, using MdlocrGetResult() with M_CHAR_SCORE. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.2.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the minimum score.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_MAX_ASPECT_RATIO	Sets the expected aspect ratio of characters to be read. This should be set when working with very wide or very narrow fonts. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 1.0.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the aspect ratio (width/height).	c		e	f	g	a		i		j	k	l	m	n	b
M_DETECTION_ANGLE	Sets the angle by which the image must be internally rotated to be able to read strings. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_REGION	Specifies to use the angle of the region of interest.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees. The angle should be specified in the counterclockwise direction.	c		e	f	g	a		i		j	k	l	m	n	b
M_DETECTION_CHAR_HEIGHT_MAX	Sets the maximum height of an uppercase character that can be read. INQ	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 150.0.	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= M_DETECTION_CHAR_HEIGHT_MIN	Specifies the maximum character height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_DETECTION_CHAR_HEIGHT_MIN		Sets the minimum height of an uppercase character that can be read. Note, setting this to a low value negatively impacts performance. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 20.0.	c		e	f	g	a		i		j	k	l	m	n	b
	6.0 <= *Value* <= M_DETECTION_CHAR_HEIGHT_MAX	Specifies the minimum character height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCHAR_MAX_WIDTH		Sets the maximum gap size between the last character of a string and the first character of another. Gaps larger than this will be read as separate strings. Adjusting this setting allows you to split and merge strings on the same line. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 50.0.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum character gap, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESEARCH_ANGLE_RANGE		Sets the range of angles at which to presearch. Note, this setting only has an impact if M_PRESEARCH_STRATEGY is set to M_PRESEARCH_NET1_V1. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NARROW_RANGE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NARROW_RANGE	Specifies to use a narrow range of [-90,90] degrees around the detection angle.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WIDE_RANGE	Specifies to use a wide range of [-180,180] degrees around the detection angle. This can decrease the speed of read operations.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESEARCH_STRATEGY		Sets which presearch algorithm to use before other MdlocrRead() operations. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_NO_PRESEARCH.	c		e	f	g	a		i		j	k	l	m	n	b

	M_NO_PRESEARCH	Specifies to not use the AI presearch strategy.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PRESEARCH_NET1_V1	Specifies to use the AI presearch strategy. The AI presearch is designed to accelerate calls to MdlocrRead() in large images containing mostly 'background' and few characters. The AI presearch also allows text to be detected at a wider angle than what would typically be possible with without the presearch.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_NUMBER		Sets the total number of strings to read in the image. For values other than M_INFINITE, using MdlocrRead() with M_MODEL_BASED will attempt to read exactly that number of strings while respecting M_MIN_NUMBER_OF_OCCURRENCES and M_MAX_NUMBER_OF_OCCURRENCES of each string model. If MdlocrRead() does not read enough strings, nothing will be read. Once the number of string occurrences for the context has been reached, the read operation will stop, regardless of whether the actual number set for any individual model has been found. Note, this setting is ignored when performing an M_READ_ALL operation. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INFINITE	Specifies that there is no limit.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of strings to read in an image.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_MAX_SIZE_X		Sets the maximum width of a target image buffer used for a Deep Learning OCR function. Note that when working with an image buffer with an ROI, M_TARGET_MAX_SIZE_X refers to the width of the ROI rather than the width of the image. Note, setting this much larger than the target image can reduce performance. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b

	M_INVALID	Specifies that the width is invalid. An error is generated if a Deep Learning OCR function is called on a context with M_TARGET_MAX_SIZE_X set to M_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the maximum width of the buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_MAX_SIZE_Y		Sets the maximum height of a target image buffer used for a Deep Learning OCR function. Note that when working with an image buffer with an ROI, M_TARGET_MAX_SIZE_Y refers to the height of the ROI rather than the height of the image. Note, setting this much larger than the target image can reduce performance. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INVALID	Specifies that the height is invalid. An error is generated if a Deep Learning OCR function is called on a context with M_TARGET_MAX_SIZE_Y set to M_INVALID.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the maximum height of the buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT		Sets the maximum read time. No results are returned if the read operation times out. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 10000.0 msec.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DISABLE	Specifies an infinite amount of read time.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0.0	Specifies the maximum read time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

The following ControlType and corresponding ControlValue parameter settings are used to control the predict engine.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling the predict engine																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_PREDICT_ENGINE		Sets the predict engine to use. You can get predict engine information by calling MdlocrInquire() with M_PREDICT_ENGINE_.... You must have installed the appropriate Aurora Imaging Library add-ons and updates to perform GPU accelerated prediction using CUDA or OpenVINO. INQ															
	M_DEFAULT	Specifies the default predict engine, set using the **Aurora Imaging Configurator** utility.	c		e	f	g	a		i		j	k	l	m	n	b
	0 <= *Value* < M_NUMBER_OF_PREDICT_ENGINES	Specifies the index of the predict engine to use.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control the Deep Learning OCR read result buffer specified with the ContextOrResultDlocrId parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a Deep Learning OCR read result buffer																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_RESULT_OUTPUT_UNITS		Sets whether to return results in pixels or world units. This essentially sets the output coordinate system to use. The setting of this control type will only affect functions within this module which return positional results. This control type can be changed at any time to return results in the required output units. INQ															
	M_DEFAULT	Same as M_ACCORDING_TO_CALIBRATION.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b

	M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. If world units are specified, calling MdlocrGetResult() generates an error if the result was not calculated on a calibrated image.	c		e	f	g	a		i		j	k	l	m	n	b
M_STOP_READ		Stops the current read operation. You can only perform this operation from another thread of higher priority. No results are returned if you stop the read operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control the Deep Learning OCR fine-tuning context specified with the ContextOrResultDlocrId parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a Deep Learning OCR fine-tuning context																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_FINETUNING_LEVEL		Sets the fine-tuning level. The fine-tuning level determines how much the fine-tuning context will adapt to the dataset. INQ															
	M_DEFAULT	Specifies the default behavior. Same as M_LOW.	c		e	f	g	a		i		j	k	l	m	n	b
	M_HIGH	Specifies a high fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LOW	Specifies a low fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_HIGH	Specifies a very high fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_LOW	Specifies a very low fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b

M_MINI_BATCH_SIZE		Sets the number of images in the fine-tuning dataset that can be processed at the same time. Note that higher values will use more memory and lower values will increase the speed of fine-tuning. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Specifies the default value; the default value is 4.	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of images in the mini-batch.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter settings are used to control the Deep Learning OCR fine-tuning result buffer specified with the ContextOrResultDlocrId parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a Deep Learning OCR fine-tuning result buffer																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_STOP_FINETUNING		Stops the current fine-tuning operation. You can only perform this operation from another thread of higher priority. No results are returned if you stop the fine-tuning operation.															
	M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

The following ControlType and corresponding ControlValue parameter setting is used to control the Deep Learning OCR read result buffer, fine-tuning result buffer, or dataset specified with the ContextOrResultDlocrId parameter.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling a Deep Learning OCR read result buffer, fine-tuning result buffer, or dataset																	
ControlType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ControlValue																
M_RESET		Removes all images and results from the Deep Learning OCR read result buffer, fine-tuning result buffer, or dataset. This does not delete the object's identifier, as is the case with MdlocrFree().															
	M_DEFAULT	Specifies the default behavior.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.

DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrControlConstraint

Synopsis

Control a positional constraint of a Deep Learning OCR string model.

Syntax

```
void MdlocrControlConstraint(
    AIL_ID ContextDlocrId,             //out
    AIL_INT64 StringModelLabelOrIndex, //in
    AIL_INT64 PositionInString,        //in
    AIL_INT64 ControlType,             //in
    AIL_DOUBLE ControlValue,           //in
    const void *ControlValuePtr        //in
)
```

Description

This function allows you to control a positional constraint of a Deep Learning OCR string model. Constraints restrict the characters permitted to be read. The Deep Learning OCR module will only read strings that follow the positional constraints. This is useful, for example, when reading serial numbers that all have the same character at a given position. Setting positional constraints will also improve performance when characters are visually similar (for example, the numeral "0" and the uppercase letter "O").

You must preprocess the Deep Learning OCR context after modifying its positional constraints and before calling MdlocrRead(). To know if a context needs to be preprocessed, call MdlocrInquire() with M_PREPROCESSED.

Parameters

ContextDlocrId

Specifies the identifier of the Deep Learning OCR context that contains the string model to control. The context must have been previously allocated on the system using MdlocrAlloc().

StringModelLabelOrIndex

Specifies the string model to control. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string model																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STRING_INDEX(AIL_INT IndexValue)	Specifies to control the string model by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Specifies the index of the string model.															

			0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the string model to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL(AIL_INT LabelValue)	Specifies to control the string model by indicating its label.			c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																
		LabelValue Specifies the label of the string model.																
			0 < *Value* < 2097152 Specifies the label of the string model to control.	c		e	f	g	a		i		j	k	l	m	n	b

PositionInString

Specifies the position of the character in the string model. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the position in a string model																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies all positions that have not been manually constrained.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the position in the string model.	c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the type of setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the setting.

See the Parameter associations section for possible values that can be specified.

ControlValuePtr

Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_CONST_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- M_NULL

Specifies the address which contains more information about the setting's new value.

Set this parameter to **M_NULL** if not used. If you use this parameter, the ControlType parameter must be M_CONSTRAINT_TYPE and the ControlValue parameter must be M_CHAR_LIST.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ControlType**, **ControlValue**, and **ControlValuePtr** parameters.

- For controlling positional constraints in a string model

The following ControlType and corresponding ControlValue and ControlValuePtr parameter settings are used to control positional constraint of a string model. Unless otherwise specified, set the StringModelLabelOrIndex parameter to the label or index of a string model, set the PositionInString parameter to the position that you want to constrain, and set the ControlValuePtr parameter to **M_NULL**.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For controlling positional constraints in a string model																	
ControlType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ControlValue																
ControlValuePtr																
M_CONSTRAINT_TYPE	Sets the type of constraint applied to the position. Note, the list of characters which the following character types correspond to is influenced by M_ACCEPTED_CHARS. For example if the character "4" is removed from the list of M_ACCEPTED_CHARS, then M_DIGITS will specify "012356789" rather than "0123456789". If all characters from the specified M_CONSTRAINT_TYPE are removed from M_ACCEPTED_CHARS, an error is generated. INQ															
ControlValue	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies any character.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_LIST	Specifies any character in the list passed to the ControlValuePtr parameter.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DIGITS	Specifies any digit.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LETTERS	Specifies any letter.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LETTERS_LOWERCASE	Specifies any lowercase letter.	c		e	f	g	a		i		j	k	l	m	n	b
	M_LETTERS_UPPERCASE	Specifies any uppercase letter.	c		e	f	g	a		i		j	k	l	m	n	b
	M_SAME_AS_DEFAULT_CONSTRAINT	Specifies to use the default constraint for the model using **MdlocrControlConstraint**() with PositionInString set to M_DEFAULT and M_CONSTRAINT_TYPE set to the required constraint. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
ControlValuePtr		Data type info: Data type: *Set this parameter to M_NULL* **|** *address of an AIL_CONST_TEXT_PTR [optionally, in C++: AIL_STRING]* Specifies a null-terminated string indicating the names of the characters supported at the specified position, when ControlValue is set to M_CHAR_LIST. Character names must refer to specific letters (such as 'o'), digits (such as '0'), and punctuation marks (such as '%') in the font. You cannot indicate a space when listing characters for M_CHAR_LIST. The specified string can contain one or more character names. List them all without separators. For example, to read the twenty-second, the ninth, or the third uppercase letter of the alphabet at the specified position, use the string "VIC". You can list character names in hexadecimal format beginning with "\\x". This is necessary if your data is in an ASCII format and you want Unicode characters beyond the Basic Latin range. For example, Basic Latin does not include the smiley face character; to specify it, use "\\x263A". You can also list a string of character names with mixed notation; in this case, you should also use "\\x" (for example, "VIC\\x263A").	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_OPTIONAL		Sets whether a position in a string model is optional. You should mark a model position as optional if you do not require a character to be	c		e	f	g	a		i		j	k	l	m	n	b

returned for that position in the model. If the string model has a minimum string size (M_STRING_SIZE_MIN) less than its maximum size (M_STRING_SIZE_MAX), optional positions can be skipped if their constraints cannot be met (up to a maximum of M_STRING_SIZE_MAX-M_STRING_SIZE_MIN skips).

For example, if you want to read 10-digit telephone numbers in the formats '###-###-####' and '##########', you can set the minimum string size to 10, set the maximum string size to 12, and mark positions 3 and 7 as optional. Then, set the permitted character constraints to M_DIGITS for positions 0-2, 4-6, and 8-11 and to hyphens for positions 3 and 7. If the target string is "123-456-7890", all the constraints are met, and Deep Learning OCR will return "123-456-7890". If the target string is "1234567890", positions 3 and 7 are skipped because they are optional and their constraints cannot be met, and Deep Learning OCR will return "1234567890".

Note that unless you set a model position as optional, its character constraints must be met to return the string, whereas if a character in the target image does not meet the constraints, Deep Learning OCR will ignore the character and check if the constraints are met by the next character if M_INTERCHAR_MAX_WIDTH is set wide enough; if the distance between the previous character and the next character is less than the maximum inter-character width, the current character is ignored and the next character can be used to meet the constraints. In the example above, if you had set the model to '##########', the target string "123-456-7890" might still have been read (as "1234567890") if the maximum inter-character width was wide enough. It is always safer to include optional characters in the model and mark their positions as optional.

You can use MdlocrGetResult() with M_SKIPPED_POSITIONS to see which positions in the model were skipped. Note that a string read with a lower number of skips will be returned as a higher priority result than a string read with a higher number of skips. INQ

	ControlValue	Specifies one of the following.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the model position is not optional.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the model position is optional. > Note, this is unavailable when PositionInString is set to M_DEFAULT.	c		e	f	g	a		i		j	k	l	m	n	b
	ControlValuePtr	Data type info: Data type: *Set this parameter to M_NULL*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrControlStringModel

Synopsis

Control a global setting of a Deep Learning OCR string model.

Syntax

```
void MdlocrControlStringModel(
    AIL_ID ContextDlocrId,             //out
    AIL_INT64 StringModelLabelOrIndex, //in
    AIL_INT64 ControlType,             //in
    AIL_DOUBLE ControlValue            //in
)
```

Description

This function allows you to control a global setting of a Deep Learning OCR string model. String models represent the strings to read from a target image. Initially, any character from any font can be read.

String models are held in a Deep Learning OCR context. To add string models to a context based on a previous call to MdlocrRead(), use MdlocrDefineModelFromResult(); to add a string model without relying on a previous MdlocrRead() operation, use MdlocrDefineModel(). To inquire about global settings of string models, use MdlocrInquireStringModel().

You must preprocess the Deep Learning OCR context after you have finished modifying its string models and before calling MdlocrRead(). To know if a context needs to be preprocessed, call MdlocrInquire() with M_PREPROCESSED.

Parameters

ContextDlocrId

Specifies the identifier of the Deep Learning OCR context that contains the string model to control. The context must have been previously allocated on the system using MdlocrAlloc().

StringModelLabelOrIndex

Specifies the string model (one or all) to control. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string model																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STRING_INDEX(AIL_INT IndexValue)	Specifies to control the string model by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Specifies the index of the string model.															

			0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the string model to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL(AIL_INT LabelValue)	Specifies to control the string model by indicating its label.			c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																
		LabelValue Specifies the label of the string model.																
			0 < *Value* < 2097152 Specifies the label of the string model to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to control all string models.			c		e	f	g	a		i		j	k	l	m	n	b

ControlType

Specifies the type of setting to control.

See the Parameter associations section for possible values that can be specified.

ControlValue

Specifies the required value for the setting.

Set this parameter to **M_DEFAULT** if not used.

See the Parameter associations section for possible values that can be specified.

Parameter associations

The table below lists possible values for the **ControlType** and **ControlValue** parameters.

- For general string model settings

To control a global setting of a string model, set the ControlType parameter to one of the values below. Unless otherwise specified, set the StringModelLabelOrIndex parameter to the label or index of a string model.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For general string model settings

ControlType / ControlValue	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_CHAR_HEIGHT_MAX	Sets the maximum height of an uppercase character in the string model. Note this value must be less than or equal to the value set using MdlocrControl() with M_DETECTION_CHAR_HEIGHT_MAX.															
M_DEFAULT	Same as M_SAME_AS_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME_AS_CONTEXT	Specifies to use the maximum character height specified using MdlocrControl() with M_DETECTION_CHAR_HEIGHT_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= M_CHAR_HEIGHT_MIN	Specifies the maximum character height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_HEIGHT_MIN	Sets the minimum height of an uppercase character in the string model. Note, this value must be greater than or equal to the value set using MdlocrControl() with M_DETECTION_CHAR_HEIGHT_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_SAME_AS_CONTEXT.	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME_AS_CONTEXT	Specifies to use the minimum character height specified using MdlocrControl() with M_DETECTION_CHAR_HEIGHT_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= M_CHAR_HEIGHT_MAX	Specifies the minimum height in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_NUMBER_OF_OCCURRENCES	Sets the maximum number of occurrences of the specified string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_INFINITE.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that there is no maximum number of occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= M_MIN_NUMBER_OF_OCCURRENCES	Specifies the maximum number of occurrences.	c		e	f	g	a		i		j	k	l	m	n	b

M_MIN_NUMBER_OF_OCCURRENCES	Sets the minimum number of occurrences of the specified string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the default value; the default value is 0.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= M_MAX_NUMBER_OF_OCCURRENCES	Specifies the minimum number of occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL_VALUE	Sets the label of the specified string model. You must set the StringModelLabelOrIndex parameter to the index of the string model.	c		e	f	g	a		i		j	k	l	m	n	b
0 < *Value* < 2097152	Specifies the label of the string model to control.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MAX	Sets the maximum number of characters in a string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MIN <= *Value* <= 256	Specifies the maximum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MIN	Sets the minimum number of characters in a string. Note, when setting M_STRING_SIZE_MIN less than M_STRING_SIZE_MAX, MdlocrRead() operations with M_MODEL_BASED will be allowed to skip model positions where M_IS_OPTIONAL is set to M_TRUE. MdlocrPreprocess() expects the number of positions with M_IS_OPTIONAL set to M_TRUE to be greater than or equal to the difference between M_STRING_SIZE_MAX and M_STRING_SIZE_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= M_STRING_SIZE_MAX	Specifies the minimum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_ANCHOR_MODE	Sets the text anchoring mode. If a text anchor is used, it will not be included in the string result.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Same as M_DISABLE.	c		e	f	g	a		i		j	k	l	m	n	b

	M_DISABLE	Specifies that no anchoring is used.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TEXT_PREFIX	Specifies that the string is prefixed by a known substring. Specify the substring using M_TEXT_ANCHOR_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TEXT_SUFFIX	Specifies that the string is suffixed by a known substring. Specify the substring using M_TEXT_ANCHOR_VALUE.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_ANCHOR_VALUE		Sets the text to be used as a textual anchor. INQ	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("String") [1]	Specifies the text used as an anchor. An empty string indicates that any string that fits the string model can be read. The default value is "".	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrCopy

Synopsis

Copy settings between two Deep Learning OCR read contexts, or copy results from a Deep Learning OCR fine-tuning result buffer to a read context.

Syntax

```
void MdlocrCopy(
    AIL_ID SrcDlOcrContextOrResult, //in
    AIL_ID DstDlOcrContext,         //out
    AIL_INT64 CopyType,             //in
    AIL_INT64 ControlFlag           //in
)
```

Description

This function copies settings from the source read context (SrcDlOcrContextOrResult) to the destination read context (DstDlOcrContext). This is useful, for example, if you want to allocate a new context using a different deep learning model (ContextType) and keep the same settings.

Alternatively, this function copies results from a source Deep Learning OCR fine-tuning result buffer (SrcDlOcrContextOrResult) to a destination read context (DstDlOcrContext). You can then use this Deep Learning OCR read context in a MdlocrRead() operation.

You must preprocess the destination Deep Learning OCR context before calling MdlocrRead() or MdlocrFinetune(). To know if a context needs to be preprocessed, call MdlocrInquire() with M_PREPROCESSED.

Parameters

SrcDlOcrContextOrResult

Specifies the identifier of the source Deep Learning OCR read context or fine-tuning result buffer from which to copy.

DstDlOcrContext

Specifies the identifier of the destination Deep Learning OCR context in which to copy.

CopyType

Specifies the type of copy operation to perform.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the copy operation																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_ALL_COMMON_SETTINGS	Specifies to copy all common settings from the source read context to the destination read context. The SrcDlOcrContextOrResult parameter must be set to a valid Deep Learning OCR read context identifier.	c		e	f	g	a		i		j	k	l	m	n	b

M_FINETUNED_CONTEXT	Specifies to copy the result of a fine-tuning operation from a source fine-tuning result buffer to a destination read context. The SrcDlOcrContextOrResult parameter must be set to a valid Deep Learning OCR fine-tuning result buffer identifier.	c		e	f	g	a		i		j	k	l	m	n	b	

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrDefineModel

Synopsis

Add a string model to, or delete a string model from, a Deep Learning OCR context.

Syntax

```
void MdlocrDefineModel(
    AIL_ID ContextDlocrId,             //in
    AIL_INT64 StringModelLabelOrIndex, //in
    AIL_INT64 Operation,               //in
    AIL_INT64 StringSizeMin,           //in
    AIL_INT64 StringSizeMax            //in
)
```

Description

This function allows you to add a string model to, or delete a string model from, a Deep Learning OCR context.

You don't need to add a model to read all strings in the target image. In addition, if you want to restrict the string that is found and read, you should typically use MdlocrDefineModelFromResult(), which allows you to select a resulting string as the model. **MdlocrDefineModel**() is useful if you only want to restrict the number of characters in the string. Regardless of how you add a model, you can always constrain the characters that appear at certain positions using MdlocrControlConstraint() and adjust general string model settings, such as, setting text anchors, using MdlocrControlStringModel().

Note that although you need to define the minimum and maximum number of characters of the string model when you add a new model with this function, you can change these limits using MdlocrControlStringModel() with M_STRING_SIZE_MIN and M_STRING_SIZE_MAX.

Parameters

ContextDlocrId

Specifies the identifier of the Deep Learning OCR context in which to add or delete a model. The Deep Learning OCR context must have been previously allocated on the required system using MdlocrAlloc().

StringModelLabelOrIndex

Specifies the string model (one or all) to control.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies whether to add or delete a string model from a Deep Learning OCR context.

See the Parameter associations section for possible values that can be specified.

StringSizeMin

Specifies the minimum number of characters of a new string model. If the Operation parameter is set to M_DELETE, StringSizeMin must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the minimum number of characters

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that this parameter is not needed (that is, when Operation set to M_DELETE).	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the minimum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b

StringSizeMax

Specifies the maximum length of a new string model. If the Operation parameter is set to M_DELETE, StringSizeMax must be set to M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the maximum number of characters																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that this parameter is not needed (that is, when Operation set to M_DELETE).	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the maximum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b

Parameter associations

The table below lists possible values for the **StringModelLabelOrIndex** and **Operation** parameters.

- For creating or deleting a string model

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For creating or deleting a string model																	
Operation		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	StringModelLabelOrIndex																
M_ADD		Specifies to add a new string model to the context.															
	M_DEFAULT	Same as M_NEW_LABEL.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STRING_LABEL(AIL_INT LabelValue)	Specifies to add a string model with the specified label.	c		e	f	g	a		i		j	k	l	m	n	b
		Parameters															
		LabelValue Specifies the label of the string model.															

			0 < *Value* < 2097152 Specifies the label of the string model to add.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NEW_LABEL	Specifies to add a string model with an automatically assigned label. To inquire about the label, use MdlocrInquireStringModel() with M_STRING_LABEL_VALUE.		c		e	f	g	a		i		j	k	l	m	n	b
M_DELETE		Specifies to delete a string model from a Deep Learning OCR context.		c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_ALL.		c		e	f	g	a		i		j	k	l	m	n	b
	M_STRING_INDEX(AIL_INT IndexValue)	Specifies to delete the string model by indicating its index.		c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																
		IndexValue Specifies the index of the string model.																
			0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the string model to delete.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STRING_LABEL(AIL_INT LabelValue)	Specifies to delete the string model by indicating its label.		c		e	f	g	a		i		j	k	l	m	n	b
		Parameters																
		LabelValue Specifies the label of the string model.																

				0 < *Value* < 2097152	Specifies the label of the string model to delete.	c		e	f	g	a		i		j	k	l	m	n	b
	M_ALL			Specifies to delete all string models from the Deep Learning OCR context.		c		e	f	g	a		i		j	k	l	m	n	b

Return value

Reserved for future expansion and returns **M_NULL**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrDefineModelFromResult

Synopsis

Add a string model based on a resulting string from a previous call to MdlocrRead().

Syntax

```
void MdlocrDefineModelFromResult(
    AIL_ID ContextDlocrId,             //in
    AIL_INT64 StringModelLabelOrIndex, //in
    AIL_ID ResultId,                   //in
    AIL_INT64 StringIndex,             //in
    AIL_INT64 ControlFlag              //in
)
```

Description

This function allows you to add a string model based on a resulting string from a previous call to MdlocrRead().

The string model is automatically assigned the same number of characters (M_STRING_SIZE_...) as the specified resulting string. The string model is also automatically assigned a minimum and maximum character height (M_CHAR_HEIGHT_...) that is the same as the specified resulting string plus a tolerance. To have the constraints inferred from the resulting string as well, set the ControlFlag parameter to M_INFER_CONSTRAINTS. Note that the position of the specified resulting string is not used.

You don't need to add a model to read all strings in the target image. You only need to add a model if you want to restrict the string that is found and read. **MdlocrDefineModelFromResult**() allows you to define the string model so that attributes are automatically set from the specified resulting string. If you only want to restrict the number of characters in the string, you can use MdlocrDefineModel(). Regardless of how you add a model, you can always constrain the characters that appear at certain positions using MdlocrControlConstraint() and adjust general string model settings using MdlocrControlStringModel().

The string model index starts at 0, and each subsequent string model added to the context is given a sequential index number, in the order that the model was added. If a model is deleted, all entries with higher indices are shifted down one. When adding the string model, you can overwrite an existing model by specifying its label or index.

If multiple strings are present in the target image, you can establish which is the required string by drawing the results using MdlocrDraw().

Parameters

ContextDlocrId

Specifies the identifier of the Deep Learning OCR context to which the string model must be added.

StringModelLabelOrIndex

Specifies the string model to replace. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string model																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Same as M_ADD.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_INDEX(AIL_INT IndexValue)	Specifies the index of the string model to replace. An error is generated if a string model is not defined at the specified index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Specifies the index of the string model.															
	0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL(AIL_INT LabelValue)	Specifies the label of the string model to replace. An error is generated if a string model is not defined at the specified label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Specifies the label of the string model.															
	0 < *Value* < 2097152 Specifies the label of the string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_ADD	Specifies to add the new string model.	c		e	f	g	a		i		j	k	l	m	n	b

ResultId

Specifies the identifier of the Deep Learning OCR result context from which the string model will be created.

StringIndex

Specifies the index of the string occurrence from which the string model will be created.

ControlFlag

Specifies what is set when defining a string model from a result. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying what is set when defining a string model from a result string																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

M_DEFAULT	Specifies to set M_STRING_SIZE_MIN, M_STRING_SIZE_MAX, M_CHAR_HEIGHT_MIN, and M_CHAR_HEIGHT_MAX for the new model based on the string occurrence.	c		e	f	g	a		i		j	k	l	m	n	b
M_INFER_CONSTRAINTS	Specifies to set constraints based on the specified string occurrence. M_LETTERS_UPPERCASE, M_LETTERS_LOWERCASE, and M_DIGITS can be inferred from the string occurrence. If a character in the string occurrence is neither a letter or a digit, then the inferred constraint type is set to M_SAME_AS_DEFAULT_CONSTRAINT (same as M_ANY).	c		e	f	g	a		i		j	k	l	m	n	b

Return value

Reserved for future expansion and returns **M_NULL**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrDeleteLastImageFromDataset

Synopsis

Delete the last image added to the Deep Learning OCR dataset.

Syntax

```
void MdlocrDeleteLastImageFromDataset(
    AIL_ID DlocrDatasetId, //out
    AIL_INT64 ControlFlag  //in
)
```

Description

This function deletes the last image and associated results that were added to a Deep Learning OCR dataset using MdlocrAddImageToDataset().

Parameters

DlocrDatasetId

Specifies the identifier of the Deep Learning OCR dataset from which to delete the image and results.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrDraw

Synopsis

Draw specific features of a Deep Learning OCR result.

Syntax

```
void MdlocrDraw(
    AIL_ID ContextGraId,           //in
    AIL_ID ResultDlocrId,          //in
    AIL_ID DstImageBufOrListGraId, //out
    AIL_INT64 Operation,           //in
    AIL_INT64 StringIndex,         //in
    AIL_INT64 CharIndex,           //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function draws specific features of a Deep Learning OCR read result in the destination image buffer or 2D graphics list.

Parameters

ContextGraId

Specifies the identifier of the 2D graphics context to use when drawing. Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the 2D graphics context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies that the default 2D graphics context of the current Aurora Imaging Library application is used. Note that there is a different default 2D graphics context for each thread.	c		e	f	g	a		i		j	k	l	m	n	b
2D graphics context identifier	Specifies a valid 2D graphics context identifier, which you have allocated using MgraAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ResultDlocrId

Specifies the identifier of the Deep Learning OCR read result buffer from which to extract the features to draw. The result buffer must have been previously allocated on the system using MdlocrAllocResult().

DstImageBufOrListGraId

Specifies the identifier of the destination image buffer or 2D graphics list in which to draw. The buffer can be any valid 1- or 3-band image buffer allocated using MbufAlloc...(). The 2D graphics list must be previously allocated using MgraAllocList(). By drawing into a display's overlay buffer or associating the 2D graphics list with the display, you can also annotate an image non-destructively.

If specifying an image buffer, it must not have a region of interest (ROI) associated with it. Using an image buffer with an ROI will cause an error.

Operation

Specifies the drawing operation to perform. When applicable, **MdlocrDraw**() draws the required information at the location that the string was read in the target, at the correct angle, scale, and aspect ratio.

For a drawing operation related to strings, you can set the Operation parameter to one of the following. The CharIndex parameter must be set to M_DEFAULT. Use the StringIndex parameter to specify the string occurrence for which to perform the draw operation. StringIndex can also be set to M_GENERAL when setting the Operation to M_DRAW_STRING_BOX.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a drawing operation related to strings																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_STRING	Draws the string that was read.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_BOX	Draws a bounding box around the string that was read. This bounding box encloses all of the string's characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_INDEX	Draws the index associated with the string read. This drawing operation is useful when defining a model using MdlocrDefineModelFromResult().	c		e	f	g	a		i		j	k	l	m	n	b

For a drawing operation related to characters, you can set the Operation parameter to one of the following. The CharIndex parameter must be set to one or all characters.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For a drawing operation related to characters																
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DRAW_CHAR	Draws the character that was read.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_CHAR_BOX	Draws the character's bounding box at the location at which the character was read.	c		e	f	g	a		i		j	k	l	m	n	b
M_DRAW_STRING_CHAR_POSITION	Draws a circled dot at the center of each bounding box at the location at which the character was read.	c		e	f	g	a		i		j	k	l	m	n	b

StringIndex

Specifies the string result (one or all) for which to perform the drawing operation. Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string index																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to perform the drawing operation for all string results.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to perform a global drawing operation for a single bounding box that encompasses all strings. The Operation parameter must be set to M_DRAW_STRING_BOX; otherwise, an error is generated.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the string result for which to perform the drawing operation.	c		e	f	g	a		i		j	k	l	m	n	b

CharIndex

Specifies the character (one or all) in the string result for which to perform the drawing operation. Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the character index																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Implements the default behavior. For drawing operations related to strings, the default indicates that this parameter is not required. For drawing operations related to characters, the default is the same as setting the M_INDEX_IN_STRING() macro to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
M_INDEX_IN_STRING(AIL_INT CharIndex)	Specifies the character(s) for which to perform the drawing operation.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	CharIndex Set this parameter to one of the following:															
	M_ALL Specifies to perform the drawing operation for all characters in the string. To retrieve the number of characters in the string, call MdlocrGetResult() with M_STRING_CHAR_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b

			0 <= *Value* < M_STRING_CHAR_NUMBER	Specifies the index of the character in the string for which to perform the drawing operation. The value must be less than the total number of characters in the string (M_STRING_CHAR_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrExport

Synopsis

Export a Deep Learning OCR dataset to disk.

Syntax

```
void MdlocrExport(
    AIL_CONST_TEXT_PTR FolderPath, //in
    AIL_INT64 DatasetFormat,       //in
    AIL_ID DlocrDatasetId,         //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function exports a Deep Learning OCR dataset to disk. This function creates a copy of all of the images in the dataset and creates a JSON file containing information about the dataset in the same folder. To import data from a JSON file, use MdlocrImport().

Parameters

FolderPath

Specifies the path of the folder in which to export the dataset. The function handles (internally) the copying of files to the folder. The destination folder should exist and be empty.

Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the folder path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*FolderPath*") [1]	Specifies the path of the folder in which to export the dataset. You must specify the folder's drive and directory (for example, "*E:\mydirectory*"). To specify a dataset folder on a remote computer (under Distributed Aurora Imaging Library), prefix the folder path string with "*remote:///*" (for example, "*remote:///E:\mydirectory*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

DatasetFormat

Reserved for future expansion and must be set to **M_DEFAULT**.

DlocrDatasetId

Specifies the identifier of the dataset to export. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the dataset to export																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
DlocrDatasetId	Specifies the identifier of a Deep Learning OCR dataset. This dataset must have been previously allocated using MdlocrAllocDataset() and contain entries added using MdlocrAddImageToDataset().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrFinetune

Synopsis

Fine-tune a Deep Learning OCR read context on a dataset.

Syntax

```
void MdlocrFinetune(
    AIL_ID FinetuneContextDlocrId, //in
    AIL_ID ReadContextDlocrId,     //in
    AIL_ID DlocrDatasetId,         //in
    AIL_ID FinetuneResultDlocrId,  //out
    AIL_INT64 ControlFlag          //in
)
```

Description

This function uses a Deep Learning OCR dataset to fine-tune a Deep Learning OCR read context. Fine-tuning results are stored in a Deep Learning OCR result buffer. To get the results, use MdlocrGetResult().

The fine-tuning process is time-consuming and heavily relies on the accuracy of the dataset's annotations. That is, the extent to which fine-tuning improves the read operation depends on the image quality of the characters or strings in the source dataset. Training time can vary considerably depending on the complexity of the images, the available hardware, and the accuracy required. Images can be added to the dataset, using MdlocrAddImageToDataset(). You can remove the last image that was added to the dataset, using MdlocrDeleteLastImageFromDataset(). You can export a dataset to the disk, using MdlocrExport() or restore a dataset from the disk, using MdlocrImport().

Before performing a fine-tuning operation, the Deep Learning OCR read and fine-tuning contexts must have been previously allocated, using MdlocrAlloc() and be in a preprocessed state (MdlocrPreprocess()).

You can manually stop the fine-tuning operation, using MdlocrControl() with.

Parameters

FinetuneContextDlocrId

Specifies the identifier of the Deep Learning OCR fine-tuning context to use for the fine-tuning operation. The context must have been previously allocated on the required system using MdlocrAlloc(), and preprocessed using MdlocrPreprocess().

ReadContextDlocrId

Specifies the identifier of the Deep Learning OCR read context to use for the fine-tuning operation. The context must have been previously allocated on the required system using MdlocrAlloc(), and preprocessed using MdlocrPreprocess().

DlocrDatasetId

Specifies the identifier of the Deep Learning OCR dataset to use for the fine-tuning operation. The dataset must have been previously allocated on the required system using MdlocrAllocDataset().

FinetuneResultDlocrId

Specifies the identifier of the Deep Learning OCR fine-tuning result buffer in which to write the results of the fine-tuning operation. The fine-tuning result buffer must have been previously allocated on the required system using MdlocrAllocResult().

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrFree

Synopsis

Free a Deep Learning OCR object.

Syntax

```
void MdlocrFree(
    AIL_ID DlocrId //in
)
```

Description

This function deletes the specified Deep Learning OCR context, Deep Learning OCR result buffer, or Deep Learning OCR dataset identifier, and releases any memory associated with it.

All Deep Learning OCR objects allocated on a particular system must be freed before the system can be freed, unless M_UNIQUE_ID was specified during allocation.

Parameter

DlocrId

Specifies the identifier of the Deep Learning OCR object to free. These must have been successfully allocated (with MdlocrAlloc(), MdlocrAllocResult(), or MdlocrAllocDataset()) prior to calling this function.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrGetHookInfo

Synopsis

Get information about a hooked Deep Learning OCR object event.

Syntax

```
AIL_INT64 MdlocrGetHookInfo(
    AIL_ID EventId,     //in
    AIL_INT64 InfoType, //in
    void *UserVarPtr    //out
)
```

Description

This function retrieves information about the event that caused the hook-handler function to be called. This function should only be called within the scope of a Deep Learning OCR object hook-handler function call (see MdlocrHookFunction()).

Parameters

EventId

Specifies the Deep Learning OCR object event identifier received by the hook-handler function. See MdlocrHookFunction() for more information. The event will be of the type passed to the HookType parameter of the hook-handler function.

See the Parameter associations section for possible values that can be specified.

InfoType

Specifies the type of information to get.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Note that when getting parameter values, the UserVarPtr parameter should be of the same data type as value of the selected parameter.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **EventId** and **InfoType** parameters and possible values returned to the **UserVarPtr** parameter.

- For getting information about a hooked Deep Learning OCR object event

j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting information about a hooked Deep Learning OCR object event																	
EventId	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
InfoType																	
UserVarPtr - Possible values returned																	
An M_FINETUNE_PROGRESS event ID	Specifies that the event that called the hook-handler function was due to an M_FINETUNE_PROGRESS event type on a fine-tuning context object.																
M_PERCENTAGE +	Retrieves the completion percentage of the fine-tuning operation.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 100.0	Specifies the percentage.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values of the InfoType parameter listed in For getting information about a hooked Deep Learning OCR object event.

You can add the following value to the above-mentioned value to determine whether an event result is available to be returned.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether event results are available to be returned																
InfoType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_AVAILABLE	Retrieves whether the requested information is available for retrieval.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_FALSE	Specifies that the requested information is not available.	c		e	f	g	a		i		j	k	l	m	n	b
M_TRUE	Specifies that the requested information is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values of the InfoType parameter listed in For getting information about a hooked Deep Learning OCR object event.

You can add one of the following values to the above-mentioned value to cast the requested information to a required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																		
InfoType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE		Casts the requested information to an *AIL_DOUBLE*.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID		Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT		Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32		Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64		Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is **M_NULL** if successful. If the operation fails, a non-null (**M_NULL**) value is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrGetResult

Synopsis

Get results from a Deep Learning OCR result buffer.

Syntax

```
void MdlocrGetResult(
    AIL_ID ResultDlocrId,        //in
    AIL_INT64 StringIndex,       //in
    AIL_INT64 CharPositionIndex, //in
    AIL_INT64 ResultType,        //in
    void *ResultArrayPtr         //out
)
```

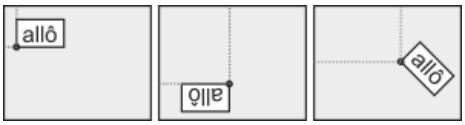
Description

This function retrieves results of the specified type from a Deep Learning OCR result buffer. You can retrieve global results, string results, or character results. Results are available after calling MdlocrRead().

If your target image (MdlocrRead()) was associated with a camera calibration context, positional and dimensional results are, by default, returned with respect to the relative coordinate system of the image. Otherwise, these results are returned in pixels, relative to the center of upper-left pixel in the target image.

If your target image was associated with a camera calibration context but you want to retrieve positional and dimensional results in pixel units, use MdlocrControl() with the M_RESULT_OUTPUT_UNITS control type set to M_PIXEL. However, if you set M_RESULT_OUTPUT_UNITS to M_WORLD without specifying a calibrated image in which to calculate the results, **MdlocrGetResult**() generates an error.

Positional and dimensional results take into account the angle at which the result was found. The following example shows how the result for the bottom-left corner of the string box (M_STRING_BOX_BL_X and M_STRING_BOX_BL_Y) depends on the location and angle at which the string was read.



To remove all results from a result buffer, call MdlocrControl() with M_RESET. This does not delete the result buffer identifier, as is the case with MdlocrFree().

Parameters

ResultDlocrId

Specifies the identifier of the Deep Learning OCR result buffer from which to retrieve results.

StringIndex

Specifies the string (one or all) for which to get results, or specifies to get global results. Set this parameter to one of the values below.

Note that you must set this parameter to a string index (one or all) when getting character results.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string for which to get results, or for specifying global results

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_ALL	Specifies to retrieve results for all strings in the result buffer. To retrieve the number of strings in the result buffer, use M_STRING_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to retrieve global results. These results are related to the result buffer itself.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_STRING_NUMBER	Specifies the index of the string for which to retrieve results. The index must be less than the total number of strings in the result buffer (M_STRING_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b

CharPositionIndex

Specifies the character (one or all) for which to get results, or specifies to get global or string results. Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the character for which to get results, or for specifying global or string results

Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Same as M_GENERAL.	c		e	f	g	a		i		j	k	l	m	n	b
M_INDEX_IN_STRING(AIL_INT CharIndex)	Specifies to retrieve a character result by indicating the character's index, according to the string model definition. The StringIndex parameter must be set to a string (one or all). Note that indices don't include any substring set as a text anchor (using MdlocrControlStringModel() with M_TEXT_ANCHOR_MODE) since these characters are not returned.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	CharIndex Set this parameter to one of the following:															

			M_ALL	Specifies to retrieve results for all characters. To retrieve the number of characters, use M_STRING_CHAR_NUMBER.	c		e	f	g	a		i		j	k	l	m	n	b
			0 <= *Value* < M_STRING_CHAR_NUMBER	Specifies the index of the character for which to get results. The value must be less than the total number of characters (M_STRING_CHAR_NUMBER).	c		e	f	g	a		i		j	k	l	m	n	b
M_GENERAL	Specifies to retrieve global results or string results. M_GENERAL retrieves global results when the StringIndex parameter is set to M_GENERAL. M_GENERAL retrieves string results when the StringIndex parameter is set to a string (one or all). Note that indices don't include any substring set as a text anchor (using MdlocrControlStringModel() with M_TEXT_ANCHOR_MODE).				c		e	f	g	a		i		j	k	l	m	n	b

ResultType

Specifies the type of result to retrieve.

See the Parameter associations section for possible values that can be specified.

ResultArrayPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]
- array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]
- array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]
- array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]
- array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]
- array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]
- array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]
- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_FLOAT
- AIL_ID
- AIL_INT
- AIL_INT16
- AIL_INT32
- AIL_INT64

Specifies the address in which to write results.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **ResultType** parameter and possible values returned to the **ResultArrayPtr** parameter.

- For retrieving global results
- For retrieving string results
- For retrieving global or string results
- For retrieving character results

When retrieving global results, set the ResultType parameter to the value below. In this case, set both the StringIndex and CharPositionIndex parameters to either M_GENERAL or M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving global results																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_STATUS +		Retrieves information regarding the state of the read operation.															
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_COMPLETE	Specifies that the read operation completed successfully. It is possible to read 0 strings (M_STRING_NUMBER), and have a successfully completed read operation.	c		e	f	g	a		i		j	k	l	m	n	b
	M_CURRENTLY_READING	Specifies that the read operation is currently ongoing. You can only get this status if you are retrieving it from another thread.	c		e	f	g	a		i		j	k	l	m	n	b
	M_INTERNAL_ERROR	Specifies that an unexpected error occurred.	c		e	f	g	a		i		j	k	l	m	n	b
	M_NOT_ENOUGH_MEMORY	Specifies that a memory allocation error occurred while reading.	c		e	f	g	a		i		j	k	l	m	n	b
	M_READ_NOT_PERFORMED	Specifies that the read operation has not been performed. This is the initial status.	c		e	f	g	a		i		j	k	l	m	n	b
	M_STOPPED_BY_REQUEST	Specifies that the read operation was explicitly stopped (MdlocrControl() with M_STOP_READ).	c		e	f	g	a		i		j	k	l	m	n	b
	M_TIMEOUT_REACHED	Specifies that the read operation ended because the timeout limit was reached (MdlocrControl() with M_TIMEOUT).	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_NUMBER +	Retrieves the total number of strings read. This is the total number of strings read and stored in the result buffer.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving string results, set the ResultType parameter to one of the values below. In this case, set the StringIndex parameter to the index of the string for which to get the result or M_ALL (unless otherwise specified), and set the CharPositionIndex parameter to either M_GENERAL or M_DEFAULT.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving string results																
ResultType ResultArrayPtr - Possible values returned	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERCHAR_MAX_WIDTH +	Retrieves the maximum inter-character width in the string that was read. The width is relative to output coordinate system specified with MdlocrControl() with M_RESULT_OUTPUT_UNITS.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the inter-character width.	c		e	f	g	a		i		j	k	l	m	n	b
M_SKIPPED_POSITIONS +	Retrieves the indices of the positions that were skipped in the string model. This array will only be filled if M_STRING_SIZE_MIN is different from M_STRING_SIZE_MAX; if they are equal, it means that no positions were skipped. Note that indices don't include any substring set as a text anchor (using MdlocrControlStringModel() with M_TEXT_ANCHOR_MODE) since these characters are not returned.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the position in the model that was skipped.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING +	Retrieves the string that was read. This information is returned as a regular string. It includes the name of all the characters in the string and the terminating null character. It does not include any text anchors set using MdlocrControlStringModel(). Note that this result type cannot be used when the StringIndex parameter is set to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_CHAR_NUMBER +	Retrieves the total number of characters read. This number does not count the terminating null character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_MODEL_INDEX +	Retrieves the index of the string model, in the Deep Learning OCR context, if one was used to read the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that no string model exists in the context.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the string model used to read the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_MODEL_LABEL +	Retrieves the label of the string model, in the Deep Learning OCR context, if one was used to read the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that no string model exists in the context.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the label of the string model used to read the string.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SCORE +	Retrieves the string score calculated during the read operation. The string score is the average score of the characters in the string.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the string score.	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving global or string results (one or all), set the ResultType parameter to one of the values below. In this case, set the StringIndex parameter to one or all strings, M_GENERAL, or M_DEFAULT, and set the CharPositionIndex parameter to either M_GENERAL or M_DEFAULT.

> Note when the StringIndex parameter is set to either M_GENERAL or M_DEFAULT, the result type will be in reference to the region that encompasses all strings, as opposed to the region of the specific string(s).

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving global or string results																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																	
M_STRING_ANGLE +	Retrieves the angle of the string's bounding box, or the bounding box encompassing all strings, in degrees. The angle is measured at the box's base and is relative to the output coordinate system (MdlocrControl() with M_RESULT_OUTPUT_UNITS). An angle interpreted with respect to the pixel coordinate system is always measured counter-clockwise, from the positive X-axis towards the negative Y-axis of the image. For information on the angle's direction of rotation when interpreting the angle with respect to the relative coordinate system, see the Angle convention in Aurora Imaging Library subsection of the Working with real-world units section of Chapter 28: Calibrating your camera setup.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the string score.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BL_X +	Retrieves the X-coordinate of the bottom-left corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BL_Y +	Retrieves the Y-coordinate of the bottom-left corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_BOX_BR_X +	Retrieves the X-coordinate of the bottom-right corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_BR_Y +	Retrieves the Y-coordinate of the bottom-right corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UL_X +	Retrieves the X-coordinate of the upper-left corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UL_Y +	Retrieves the Y-coordinate of the upper-left corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UR_X +	Retrieves the X-coordinate of the upper-right corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_BOX_UR_Y +	Retrieves the Y-coordinate of the upper-right corner of the bounding box of the string, or the bounding box encompassing all strings, relative to the angle at which the string was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_HEIGHT +	Retrieves the height of the string's bounding box, or the bounding box encompassing all strings.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_POSITION_X +	Retrieves the X-coordinate of the center of the bounding box of the string, or the bounding box encompassing all strings; this position is considered the position of the string, or bounding box of all strings.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_POSITION_Y +	Retrieves the Y-coordinate of the center of the bounding box of the string, or the bounding box encompassing all strings; this position is considered the position of the string, or bounding box of all strings.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_WIDTH +	Retrieves the width of the string's bounding box, or the bounding box encompassing all strings.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

When retrieving results for characters in a string, set the ResultType parameter to one of the values below. In this case, set the StringIndex parameter to a string (one or all), and set the CharPositionIndex parameter to a character (one or all using M_INDEX_IN_STRING()). Characters are ordered by their index position in their string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving character results																
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_CHAR_BOX_BL_X +	Retrieves the X-coordinate of the bottom-left corner of the character's bounding box, relative to the angle at which the character was found.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BL_Y +	Retrieves the Y-coordinate of the bottom-left corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BR_X +	Retrieves the X-coordinate of the bottom-right corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_BR_Y +	Retrieves the Y-coordinate of the bottom-right corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UL_X +	Retrieves the X-coordinate of the upper-left corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UL_Y +	Retrieves the Y-coordinate of the upper-left corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UR_X +	Retrieves the X-coordinate of the upper-right corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_BOX_UR_Y +	Retrieves the Y-coordinate of the upper-right corner of the character's bounding box, relative to the angle at which the character was found.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_HEIGHT +	Retrieves the height of the character's bounding box.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_NAME +	Retrieves the name of the character in the string that was read. The name is returned as a regular string. It consists of the character name and the terminating null character. Note that M_CHAR_NAME cannot be used when the StringIndex parameter is set to M_ALL or when the CharPositionIndex parameter is set to M_ALL.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_X +	Retrieves the X-coordinate of the center of the bounding box of the character; this position is considered the position of the character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_POSITION_Y +	Retrieves the Y-coordinate of the center of the bounding box of the character; this position is considered the position of the character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_SCORE +	Retrieves the character's score. This quantifies the confidence that the Deep Learning OCR module has correctly predicted the character.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the character score.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_WIDTH +	Retrieves the width of the character's bounding box.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* **|** *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_STRING; M_CHAR_NAME.

You can add the following value to the above-mentioned values to get the size of a string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the size of a string																
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	ResultArrayPtr - Possible values returned															
M_STRING_SIZE +		Retrieves the number of characters in the string, including the terminating null character. For M_CHAR_NAME, the name of each letter at each position is itself a string, with its own terminating null character. You would need to sum them all to get the required array size for a string. For a specific string, M_STRING + M_STRING_SIZE returns the number of characters in the string plus 1 for each character's terminating null character.														
	ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For retrieving global results
- For retrieving string results
- For retrieving global or string results
- For retrieving character results

the values listed in all the parameter association tables.

You can add the following value to the above-mentioned values to determine whether a result is available.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining whether results are available																	
ResultType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	ResultArrayPtr - Possible values returned																
M_AVAILABLE		Retrieves whether the requested result type is available for retrieval.															
	ResultArrayPtr extra info	Data type info: Data type: *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]* **|** *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the requested result type is not available.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the requested result type is available.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For retrieving string results
- For retrieving global or string results
- For retrieving character results

the values listed in all the parameter association tables except **For retrieving global results**.

You can add the following value to the above-mentioned values to determine the required array size (number of elements) to store the returned values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For determining the required array size (number of elements) to store the returned values																	
ResultType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_NB_ELEMENTS	Retrieves the required array size (number of elements) to store the returned values.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For retrieving global results, For retrieving global or string results; and for the following values: M_SKIPPED_POSITIONS; M_STRING_CHAR_NUMBER; M_STRING_MODEL_INDEX; M_STRING_MODEL_LABEL; M_STRING_SCORE; M_CHAR_BOX_BL_X; M_CHAR_BOX_BL_Y; M_CHAR_BOX_BR_X; M_CHAR_BOX_BR_Y; M_CHAR_BOX_UL_X; M_CHAR_BOX_UL_Y; M_CHAR_BOX_UR_X; M_CHAR_BOX_UR_Y; M_CHAR_HEIGHT; M_CHAR_POSITION_X; M_CHAR_POSITION_Y; M_CHAR_SCORE; M_CHAR_WIDTH; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested results to a required data type.

> Note: In C++, when using a standard vector (std::vector) instead of an array, the values listed in this combination table must not be used. If you need to use a specific data type, instantiate the vector with the necessary data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting the result to a required data type																	
ResultType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
ResultArrayPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested results to an *AIL_DOUBLE*.															
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE* ┃ *array of type AIL_DOUBLE [optionally, in C++: a reference to a std::vector<AIL_DOUBLE>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_FLOAT	Casts the requested results to an *AIL_FLOAT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_FLOAT* ┃ *array of type AIL_FLOAT [optionally, in C++: a reference to a std::vector<AIL_FLOAT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested results to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b

ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_ID* **|** *array of type AIL_ID [optionally, in C++: a reference to a std::vector<AIL_ID>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested results to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT* **|** *array of type AIL_INT [optionally, in C++: a reference to a std::vector<AIL_INT>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT16	Casts the requested results to an *AIL_INT16*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT16* **|** *array of type AIL_INT16 [optionally, in C++: a reference to a std::vector<AIL_INT16>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested results to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT32* **|** *array of type AIL_INT32 [optionally, in C++: a reference to a std::vector<AIL_INT32>]*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested results to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
ResultArrayPtr extra info	Data type info: Data type: *address of an AIL_INT64* **|** *array of type AIL_INT64 [optionally, in C++: a reference to a std::vector<AIL_INT64>]*	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrGetStringIndex

Synopsis

Get the index of a string that touches a specified image position, from a Deep Learning OCR result buffer.

Syntax

```
AIL_INT64 MdlocrGetStringIndex(
    AIL_ID ResultDlocrId,     //in
    AIL_DOUBLE PositionX,     //in
    AIL_DOUBLE PositionY,     //in
    AIL_INT64 *StringIndexPtr //out
)
```

Description

This function retrieves the index of a string that touches a specified image position, from a Deep Learning OCR result buffer. This is useful, for example, in combination with MdlocrDefineModelFromResult() to define a string model from a position on an image. You can establish the position of the string of interest by drawing the results using MdlocrDraw().

Parameters

ResultDlocrId

Specifies the identifier of the Deep Learning OCR result buffer from which to retrieve results.

PositionX

Specifies the X-position in the image at which to find a string. You can change whether to use pixel or world coordinates using MdlocrControl() with M_RESULT_OUTPUT_UNITS.

PositionY

Specifies the Y-position in the image at which to find a string. You can change whether to use pixel or world coordinates using MdlocrControl() with M_RESULT_OUTPUT_UNITS.

StringIndexPtr

Specifies the address in which to write the index of the string if one is found touching the specified position. If no string touches the specified position, **M_INVALID** is written at this address. Since **MdlocrGetStringIndex**() also returns the requested information, you can set this parameter to **M_NULL**.

Return value

The returned value is the index of the string that touches the specified position. If no string touches, **M_INVALID** is returned.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrHookFunction

Synopsis

Hook a function to an event that occurs using a Deep Learning OCR fine-tuning context.

Syntax

```
void MdlocrHookFunction(
    AIL_ID ContextDlocrId,                      //in
    AIL_INT64 HookType,                         //in
    AIL_DLOCR_HOOK_FUNCTION_PTR HookHandlerPtr, //in
    void *UserDataPtr                           //in-out
)
```

Description

This function allows you to attach or detach a user-defined function to a Deep Learning OCR fine-tuning event. Once a hook-handler function is defined and hooked to an event, it is automatically called when the event occurs.

You can hook more than one function to an event by making separate calls to **MdlocrHookFunction**() for each function that you want to hook. Aurora Imaging Library automatically chains and keeps an internal list of all these hooked functions. When a function is hooked, this new function is added to the end of the list. When the event happens, all user-defined functions in the list will be executed in the same order that they were hooked to the event. You can also remove any function from the list; in this case, Aurora Imaging Library preserves the order of the remaining functions in the list.

You can obtain information concerning the event, using MdlocrGetHookInfo(), and take appropriate action before returning control to the application.

Parameters

ContextDlocrId

Specifies the identifier of the Deep Learning OCR fine-tuning context onto which to attach a hook-handler. Set this parameter to one of the following values.

HookType

Specifies the event type. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the event type																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FINETUNE_PROGRESS +	Calls the hook-handler function each time progress is made during an MdlocrFinetune() operation.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for any of the possible values of the HookType parameter.

You can add the following value to the above-mentioned value to set whether to detach the hook-handler function.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For the HookType parameter																	
Combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_UNHOOK	Unhooks the specified function if hooked to the specified event. When you use M_UNHOOK, you must provide the same values for all the parameters as when you originally hooked the function.	c		e	f	g	a		i		j	k	l	m	n	b

HookHandlerPtr

Specifies the address of the function that should be called when an event occurs.

The hook-handler function, pointed to by HookHandlerPtr, must be declared as follows:

```
AIL_INT MFTYPE HookHandler(
    AIL_INT HookType,
    AIL_ID EventId,
    void *UserDataPtr
)
```

Parameters

HookType

Specifies the type of event that generated the call.

EventId

Event identifier to pass to MdlocrGetHookInfo() when inquiring about the hooked event.

UserDataPtr

User data pointer that was passed to the UserDataPtr parameter of **MdlocrHookFunction**().

Upon successful completion, the hook-handler function should return **M_NULL**. Note, *MFTYPE* and *AIL_DLOCR_HOOK_FUNCTION_PTR* are reserved Aurora Imaging Library predefined types for functions and data pointers, respectively.

UserDataPtr

Specifies the address of the user data that you want to make available to the hook-handler function. This address is passed to the hook-handler function, through its UserDataPtr parameter, when the specified event occurs. Set this parameter to **M_NULL** if user data is not required.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrImport

Synopsis

Import a Deep Learning OCR dataset from disk.

Syntax

```
void MdlocrImport(
    AIL_CONST_TEXT_PTR FolderPath, //in
    AIL_INT64 DatasetFormat,       //in
    AIL_INT64 Operation,           //in
    AIL_ID DlocrDatasetId,         //in
    AIL_INT64 ControlFlag          //in
)
```

Description

This function imports a Deep Learning OCR dataset from disk. This function reads the specified JSON file describing the dataset and checks that all images can be restored. To export data from a dataset, use MdlocrExport().

Parameters

FolderPath

Specifies the name and path of a file from which to import. When specifying the name and path of a file, the function handles (internally) the opening and closing of that file. Also, for easier use with other Aurora Imaging software products, the specified file name must use the JSON file extension.

Set this parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
AIL_TEXT("*JsonPath*") [1]	Specifies the required information for the file from which to import. You must specify the file's drive, directory, and name (for example, "*E:\Manual\MySweetSweetDataset.json*"). To specify a dataset file on a remote computer (under Distributed Aurora Imaging Library), prefix the file name string with "*remote:///*" (for example, "*remote:///E:\Manual\\MyEvenSweeterDataset.json*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

DatasetFormat

Reserved for future expansion and must be set to **M_DEFAULT**.

Operation

Specifies the type of import operation to perform.

Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of the file that you can import																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_APPEND	Specifies to append entries to the existing entries in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Specifies to overwrite any existing entries in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b

DlocrDatasetId

Specifies the identifier of the dataset in which to import. Set this parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the dataset in which to import																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
DlocrDatasetId	Specifies the identifier of a Deep Learning OCR dataset context. This dataset must have been previously allocated using MdlocrAllocDataset().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrInquire

Synopsis

Inquire about a Deep Learning OCR context, result buffer, or dataset setting.

Syntax

```
AIL_INT64 MdlocrInquire(
    AIL_ID DlocrId,        //in
    AIL_INT64 InquireType, //in
    void *UserVarPtr       //out
)
```

Description

This function inquires about a setting of a Deep Learning OCR context, result buffer, or dataset. Such settings are typically specified with MdlocrControl().

Parameters

DlocrId

Specifies the identifier of the Deep Learning OCR context, result buffer, or dataset about which to inquire. The context must have been previously allocated on the system using MdlocrAlloc(). The result buffer must have been previously allocated on the system using MdlocrAllocResult(). The dataset must have been previously allocated on the system using MdlocrAllocDataset().

InquireType

Specifies the setting to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_ID
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MdlocrInquire**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The tables below list possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about a Deep Learning OCR context or result buffer
- For inquiring about a Deep Learning OCR Read or Finetuning Context
- For inquiring about a Deep Learning OCR Read Context

- For inquiring about the predict engine
- For inquiring about a Deep Learning OCR fine-tuning context
- For inquiring about a Deep learning OCR read result buffer
- For inquiring about a Deep learning OCR dataset

To inquire about the Deep Learning OCR read context, finetuning context, read result buffer, or finetuning result buffer specified with the DlocrId parameter, set the InquireType parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a Deep Learning OCR context or result buffer																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_MODIFICATION_COUNT +	Inquires the current value of the modification counter. The modification counter is increased by one each time settings for the context or result buffer are modified. Although you cannot identify the modification counter's contents, you can compare the counter's value throughout your application to know if the context or result buffer has been altered. If the modification counter has changed you can, for example, prompt the user to save before closing the application.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value	Specifies the current value of the modification counter.	c		e	f	g	a		i		j	k	l	m	n	b
M_OWNER_SYSTEM +	Inquires the identifier of the system on which the context or result buffer was allocated. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the Deep Learning OCR read or finetuning context specified with the DlocrId parameter, set the InquireType parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a Deep Learning OCR Read or Finetuning Context																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_CONTEXT_TYPE +		Inquires the type of context. SET														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_OCR_NET1_BALANCED_V1	Specifies a Deep Learning OCR read context with a balanced deep learning model based on the NET1 architecture. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_OCR_NET1_EXTENDED_V1	Specifies a Deep Learning OCR read context with the most robust deep learning model based on the NET1 architecture. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_OCR_NET1_FAST_V1	Specifies a Deep Learning OCR read context with the fastest deep learning model based on the NET1 architecture. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_OCR_NET2_FINETUNE_CONTEXT	Specifies a Deep Learning OCR fine-tuning context. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_OCR_NET2_TINY_V1	Specifies a Deep Learning OCR read context with the tiny deep learning model based on the NET2 architecture. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_PREPROCESSED +		Inquires whether the context is preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the context is not preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the context is preprocessed.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the Deep Learning OCR read context specified with the DlocrId parameter, set the InquireType parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a Deep Learning OCR Read Context																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																

M_ACCEPTED_CHARS +	Inquires the list of accepted characters. The characters in this list should be included in M_SUPPORTED_CHARS. SET														
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Accepted characters*") 1	Specifies the list of accepted characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_ACCEPTANCE +	Inquires he minimum score that a matched character must have to be considered valid. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 1.0	Specifies the minimum score.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_MAX_ASPECT_RATIO +	Inquires the maximum aspect ratio of an uppercase character. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the aspect ratio (width/height).	c		e	f	g	a		i		j	k	l	m	n	b
M_DETECTION_ANGLE +	Inquires the angle by which the image must be internally rotated to be able to read strings. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_ACCORDING_TO_REGION	Specifies to use the angle of the region of interest.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= 360.0	Specifies the angle, in degrees. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_DETECTION_CHAR_HEIGHT_MAX +	Inquires the maximum height of an uppercase character that can be read. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= M_DETECTION_CHAR_HEIGHT_MIN	Specifies the maximum character height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_DETECTION_CHAR_HEIGHT_MIN +	Inquires the minimum height of an uppercase character that can be read. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
6.0 <= *Value* <= M_DETECTION_CHAR_HEIGHT_MAX	Specifies the minimum character height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERCHAR_MAX_WIDTH +	Inquires the maximum gap size between the last character of a string and the first character of another. Gaps larger than this will be read as separate strings. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the maximum character gap, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_NUMBER_OF_STRING_MODELS +	Inquires the number of string models in the context. To add and remove string models, use MdlocrDefineModelFromResult() or MdlocrDefineModel().	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of string models.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESEARCH_ANGLE_RANGE +	Inquires the range of angles at which to presearch. Note, this setting only has an impact if M_PRESEARCH_STRATEGY is set to M_PRESEARCH_NET1_V1. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NARROW_RANGE	Specifies to use a narrow range of [-90,90] degrees around the detection angle.	c		e	f	g	a		i		j	k	l	m	n	b
M_WIDE_RANGE	Specifies to use a wide range of [-180,180] degrees around the detection angle. INFO	c		e	f	g	a		i		j	k	l	m	n	b

M_PRESEARCH_STRATEGY +	Inquires which presearch algorithm to use before other MdlocrRead() operations. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_NO_PRESEARCH	Specifies to not use the AI presearch strategy.	c		e	f	g	a		i		j	k	l	m	n	b
M_PRESEARCH_NET1_V1	Specifies to use the AI presearch strategy. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_NUMBER +	Inquires the maximum total number of strings to read in the image. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that there is no limit.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the number of strings to read in an image.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED_CHARS +	Inquires the list of supported characters. This is the list of characters on which the classifier was trained.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*Supported characters*") 1	Specifies the list of supported characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_TARGET_MAX_SIZE_X +	Inquires the maximum width of a target image buffer used for Deep Leaning OCR functions. When using ROIs, this can be set to the width of the ROI to optimize the operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the width is invalid. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum width of the buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b

M_TARGET_MAX_SIZE_Y +	Inquires the maximum height of a target image buffer used for Deep Learning OCR functions. When using ROIs, this can be set to the height of the ROI to optimize the operation. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the height is invalid. INFO	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the maximum height of the buffer, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_TIMEOUT +	Inquires the maximum read time. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies an infinite amount of read time.	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0.0	Specifies the maximum read time, in msec.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about the predict engine of the Deep Learning OCR context specified with the DlocrId parameter, set the InquireType parameter to one of the following values.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about the predict engine																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_NUMBER_OF_PREDICT_ENGINES +	Inquires the number of predict engines detected. You can inquire the specifics of each predict engine using M_PREDICT_ENGINE_PROVIDER, M_PREDICT_ENGINE_DESCRIPTION and M_PREDICT_ENGINE_PRECISION.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

	Value >= 0	Specifies the number of predict engines.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ENGINE_DESCRIPTION +		Inquires the description of the predict engine being used. You can inquire the predict engine being used with M_PREDICT_ENGINE_USED.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
	AIL_TEXT("*Description*") 1	Specifies the text description of the engine type seen by the execution provider.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ENGINE_PRECISION +		Inquires the precision of the predict engine being used. You can inquire the predict engine being used with M_PREDICT_ENGINE_USED.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FP16	Specifies that the precision used by the predict engine is floating-point 16-bit. INFO	c		e	f	g	a		i		j	k	l	m	n	b
	M_FP32	Specifies that the precision used by the predict engine is floating-point 32-bit.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ENGINE_PROVIDER +		Inquires the provider of the predict engine being used. You can inquire the predict engine being used with M_PREDICT_ENGINE_USED. You must have installed the appropriate Aurora Imaging Library add-ons and updates to perform GPU accelerated prediction using CUDA or OpenVINO.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_CUDA	Specifies that the predict engine provider is CUDA.	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT_CPU	Specifies the default predict engine provider.	c		e	f	g	a		i		j	k	l	m	n	b

M_OPENVINO	Specifies that the predict engine provider is OpenVINO.	c		e	f	g	a		i		j	k	l	m	n	b
M_PREDICT_ENGINE_USED +	Inquires which predict engine is used by the specified context. This value should match the value set using MdlocrControl() with M_PREDICT_ENGINE unless the order of the predict engines is modified. This can happen if the environment running Aurora Imaging Library is modified. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies that the predict engine being used is the predict engine that was selected in the **Aurora Imaging Configurator** utility.	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the predict engine is not found.	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* < M_NUMBER_OF_PREDICT_ENGINES	Specifies the index of the predict engine used.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

To inquire about the Deep Learning OCR fine-tuning context specified with the DlocrId parameter, set the InquireType parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a Deep Learning OCR fine-tuning context																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_FINETUNING_LEVEL +	Inquires the fine-tuning level. The fine-tuning level determines how much the fine-tuning context will adapt to the dataset. SET															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_HIGH	Specifies a high fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b

	M_LOW	Specifies a low fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b
	M_MEDIUM	Specifies a medium fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_HIGH	Specifies a very high fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b
	M_VERY_LOW	Specifies a very low fine-tuning level.	c		e	f	g	a		i		j	k	l	m	n	b
M_MINI_BATCH_SIZE +		Inquires the number of images in the fine-tuning dataset that can be processed at the same time. Note that higher values will use more memory and lower values will increase the speed of fine-tuning. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	Value > 0	Specifies the number of images in the mini-batch.	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the Deep Learning OCR read result buffer specified with the DlocrId parameter, set the InquireType parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a Deep learning OCR read result buffer																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_RESULT_OUTPUT_UNITS +		Inquires whether to return results in pixels or world units. SET															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_ACCORDING_TO_CALIBRATION	Specifies that results are returned in world units if the result was calculated on an image associated with a camera calibration context; otherwise, specifies that results are returned in pixel units.	c		e	f	g	a		i		j	k	l	m	n	b
	M_PIXEL	Specifies that results are returned in pixel units, with respect to the pixel coordinate system.	c		e	f	g	a		i		j	k	l	m	n	b
	M_WORLD	Specifies that results are returned in world units, with respect to the relative coordinate system. INFO	c		e	f	g	a		i		j	k	l	m	n	b

To inquire about the Deep Learning OCR dataset specified with the DlocrId parameter, set the InquireType parameter to one of the values below.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a Deep learning OCR dataset																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_NUMBER_OF_ENTRIES +	Inquires the number of entries in a dataset. An entry contains an image and its associated results.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the number of entries in the dataset.	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_PREDICT_ENGINE_DESCRIPTION.

You can add the following value to the above-mentioned value to get the size of a string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the size of a string																	
InquireType combination value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_STRING_SIZE +	Inquires the number of characters in the string. This number accounts for every character, including the terminating null character.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for
Tables:

- For inquiring about a Deep Learning OCR context or result buffer
- For inquiring about a Deep Learning OCR Read or Finetuning Context
- For inquiring about a Deep Learning OCR Read Context
- For inquiring about a Deep Learning OCR fine-tuning context
- For inquiring about a Deep learning OCR read result buffer

the values listed in all the parameter association tables except **For inquiring about the predict engine**, **For inquiring about a Deep learning OCR dataset**.

You can add the following value to the above-mentioned values to get the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_DEFAULT		Inquires the default value of the specified inquire type.														
	UserVarPtr extra info	Data type info: Data type: *defined elsewhere*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for
Tables:

- For inquiring about a Deep Learning OCR context or result buffer
- For inquiring about a Deep Learning OCR Read or Finetuning Context
- For inquiring about a Deep Learning OCR Read Context
- For inquiring about a Deep Learning OCR fine-tuning context
- For inquiring about a Deep learning OCR read result buffer
- For inquiring about a Deep learning OCR dataset

the values listed in all the parameter association tables except **For inquiring about the predict engine**.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported for the Deep Learning OCR context currently being inquired.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring if an inquire type is supported																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_HAS_DEFAULT		Inquires whether the specified inquire type has a default value.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED		Inquires whether the specified inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about a Deep Learning OCR context or result buffer, For inquiring about a Deep Learning OCR Read or Finetuning Context, For inquiring about a Deep Learning OCR

Read Context, For inquiring about a Deep Learning OCR fine-tuning context, For inquiring about a Deep learning OCR read result buffer, For inquiring about a Deep learning OCR dataset; and for the following values: M_NUMBER_OF_PREDICT_ENGINES; M_PREDICT_ENGINE_PRECISION; M_PREDICT_ENGINE_PROVIDER; M_PREDICT_ENGINE_USED; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the data type																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_ID	Casts the requested information to an *AIL_ID*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_ID*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT64*. If the requested information does not fit into an *AIL_INT64*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrInquireConstraint

Synopsis

Inquire about a positional constraint of a Deep Learning OCR string model.

Syntax

```
AIL_INT64 MdlocrInquireConstraint(
    AIL_ID ContextDlocrId,             //in
    AIL_INT64 StringModelLabelOrIndex, //in
    AIL_INT64 PositionInString,        //in
    AIL_INT64 InquireType,             //in
    void *UserVarPtr                   //out
)
```

Description

This function allows you to inquire about a positional constraint of a Deep Learning OCR string model.

Parameters

ContextDlocrId

Specifies the identifier of the Deep Learning OCR context that contains the string model about which to inquire. The context must have been previously allocated on the system using MdlocrAlloc().

StringModelLabelOrIndex

Specifies the string model about which to inquire. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string model																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STRING_INDEX(AIL_INT IndexValue)	Specifies to inquire about the string model by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Specifies the index of the string model.															
	0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the string model about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_LABEL(AIL_INT LabelValue)	Specifies to inquire about the string model by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Specifies the label of the string model.															
	0 < *Value* < 2097152 Specifies the label of the string model about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b

PositionInString

Specifies the position of the character in the string model. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the position in a string model																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Specifies all positions that have not been manually constrained.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the position in the string model.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_INT
- AIL_INT32
- AIL_INT64
- AIL_UINT8

Specifies the address in which to write the requested information. Since the **MdlocrInquireConstraint**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about positional constraints in a string model

The following InquireType parameter settings are used to inquire about positional constraints of a string model.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about positional constraints in a string model																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_CHAR_LIST +	Inquires the list of expected characters (case sensitive). If M_CONSTRAINT_TYPE is set to M_ANY, this will return an empty string indicating all supported values are used.															
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*CharList*") 1	Specifies the list of expected characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_CONSTRAINT_TYPE +	Inquires the type of constraint applied to the position. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_ANY	Specifies any character.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_LIST	Specifies any character in a list passed to the ControlValuePtr parameter of MdlocrControlConstraint().	c		e	f	g	a		i		j	k	l	m	n	b
M_DIGITS	Specifies any digit.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTERS	Specifies any letter.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTERS_LOWERCASE	Specifies any lowercase letter.	c		e	f	g	a		i		j	k	l	m	n	b
M_LETTERS_UPPERCASE	Specifies any uppercase letter.	c		e	f	g	a		i		j	k	l	m	n	b

	M_SAME_AS_DEFAULT_CONSTRAINT	Specifies to use the default constraint set using MdlocrControlConstraint() with its PositionInString set to M_DEFAULT and M_CONSTRAINT_TYPE set to the required constraint. This is the default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_IS_OPTIONAL +		Inquires whether a position in a string model is optional. SET	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_DEFAULT	Same as M_FALSE.	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the model position is not optional.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the model position is optional.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

Combination value for the values listed in For inquiring about positional constraints in a string model.

You can add the following value to the above-mentioned values to get the default value of an inquire type (if supported), regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_DEFAULT		Inquires the default value of the specified inquire type.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for M_CHAR_LIST.

You can add the following value to the above-mentioned value to get the size of a string.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the size of a string																	
InquireType combination value		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)

	UserVarPtr - Possible values returned															
M_STRING_SIZE +		Inquires the number of characters in the string. This number accounts for every character, including the terminating null character. All strings have the terminating null character, even though you need not explicitly list it when specifying the string.														
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about positional constraints in a string model.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported for the constraint currently being inquired.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring if an inquire type is supported																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_HAS_DEFAULT		Inquires whether the specified inquire type has a default value.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED		Inquires whether the specified inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for M_CONSTRAINT_TYPE; M_IS_OPTIONAL; M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting the requested information to the required data type

InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_UINT8	Casts the requested information to an *AIL_UINT8*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_UINT8*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT64*. If the requested information does not fit into an *AIL_INT64*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrInquireStringModel

Synopsis

Inquire about a global setting of a Deep Learning OCR string model.

Syntax

```
AIL_INT64 MdlocrInquireStringModel(
    AIL_ID ContextDlocrId,             //in
    AIL_INT64 StringModelLabelOrIndex, //in
    AIL_INT64 InquireType,             //in
    void *UserVarPtr                   //out
)
```

Description

This function inquires about a global setting of a Deep Learning OCR string model. Global settings of a string model are set using MdlocrControlStringModel().

Parameters

ContextDlocrId

Specifies the identifier of the Deep Learning OCR context that contains the string model about which to inquire. The context must have been previously allocated on the system using MdlocrAlloc().

StringModelLabelOrIndex

Specifies the string model about which to inquire. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string model																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_STRING_INDEX(AIL_INT IndexValue)	Specifies to inquire about the string model by indicating its index.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	IndexValue Specifies the index of the string model.															
	0 <= *Value* < M_NUMBER_OF_STRING_MODELS Specifies the index of the string model about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_LABEL(AIL_INT LabelValue)	Specifies to inquire about the string model by indicating its label.	c		e	f	g	a		i		j	k	l	m	n	b
	Parameters															
	LabelValue Specifies the label of the string model.															
	0 < *Value* < 2097152 — Specifies the label of the string model about which to inquire.	c		e	f	g	a		i		j	k	l	m	n	b

InquireType

Specifies the type of setting about which to inquire.

See the Parameter associations section for possible values that can be specified.

UserVarPtr
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- array of type AIL_TEXT_CHAR [optionally, in C++: a reference to an AIL_STRING]
- AIL_DOUBLE
- AIL_INT
- AIL_INT32
- AIL_INT64

Specifies the address in which to write the requested information. Since the **MdlocrInquireStringModel**() function also returns the requested information, you can set this parameter to **M_NULL**.

See the Parameter associations section for possible values that can be returned.

Parameter associations

The table below lists possible values for the **InquireType** parameter and possible values returned to the **UserVarPtr** parameter.

- For inquiring about a global setting of a string model

To inquire about a global setting of a string model, set the InquireType parameter to one of the values below. Unless otherwise specified, set the StringModelLabelOrIndex parameter to the label or index of a string model.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring about a global setting of a string model																	
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																

M_CHAR_HEIGHT_MAX +	Inquires the maximum height of an uppercase character in the string model.														
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME_AS_CONTEXT	Specifies to use the maximum character height specified using MdlocrControl() with M_DETECTION_CHAR_HEIGHT_MAX.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= M_CHAR_HEIGHT_MIN	Specifies the maximum character height, in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_CHAR_HEIGHT_MIN +	Inquires the minimum height of an uppercase character in the string model.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_SAME_AS_CONTEXT	Specifies to use the minimum character height specified using MdlocrControl() with M_DETECTION_CHAR_HEIGHT_MIN.	c		e	f	g	a		i		j	k	l	m	n	b
0.0 <= *Value* <= M_CHAR_HEIGHT_MAX	Specifies the minimum height in pixels.	c		e	f	g	a		i		j	k	l	m	n	b
M_MAX_NUMBER_OF_OCCURRENCES +	Inquires the maximum number of occurrences of the specified string model.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INFINITE	Specifies that there is no maximum number of occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= M_MIN_NUMBER_OF_OCCURRENCES	Specifies the maximum number of occurrences.	c		e	f	g	a		i		j	k	l	m	n	b
M_MIN_NUMBER_OF_OCCURRENCES +	Inquires the minimum number of occurrences of the specified string model.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
0 <= *Value* <= M_MAX_NUMBER_OF_OCCURRENCES	Specifies the minimum number of occurrences.	c		e	f	g	a		i		j	k	l	m	n	b

M_STRING_INDEX_VALUE +	Inquires the index of the string model. Set the StringModelLabelOrIndex parameter to label of the string model.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_INVALID	Specifies that the label is invalid.	c		e	f	g	a		i		j	k	l	m	n	b
Value >= 0	Specifies the index of the string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_LABEL_VALUE +	Inquires the label of the string model. Set the StringModelLabelOrIndex parameter to index of the string model.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
Value > 0	Specifies the label of the string model.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MAX +	Inquires the maximum number of characters in a string.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MIN <= *Value* <= 256	Specifies the maximum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_STRING_SIZE_MIN +	Inquires the minimum number of characters in a string.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
1 <= *Value* <= M_STRING_SIZE_MAX	Specifies the minimum number of characters.	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_ANCHOR_MODE +	Inquires the text anchoring mode.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_DISABLE	Specifies that no anchoring is used.	c		e	f	g	a		i		j	k	l	m	n	b

M_TEXT_PREFIX	Specifies that the string is prefixed by a known substring. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_SUFFIX	Specifies that the string is suffixed by a known substring. INFO	c		e	f	g	a		i		j	k	l	m	n	b
M_TEXT_ANCHOR_VALUE +	Inquires the text to be used as a textual anchor. SET	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *array of type AIL_TEXT_CHAR [optionally, in C++: AIL_STRING]*	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("String") 1	Specifies the text used as an anchor. INFO	c		e	f	g	a		i		j	k	l	m	n	b

[1] If the value was passed in a variable, it won't be returned enclosed in AIL_TEXT().

Combination value for M_TEXT_ANCHOR_VALUE.

You can add the following value to the above-mentioned value to get the string's length.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For getting the string size																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_STRING_SIZE +	Retrieves the length of the string, including the terminating null character ("\0").															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b

Combination value for the values listed in For inquiring about a global setting of a string model; (exceptions might exist) and for the following value: M_TEXT_ANCHOR_VALUE.

You can add the following value to the above-mentioned values to get the default value of an inquire type, regardless of the current value of the inquire type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring the default value																
InquireType	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
UserVarPtr - Possible values returned																
M_DEFAULT	Inquires the default value of the specified inquire type.															

	UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about a global setting of a string model; (exceptions might exist) and for the following value: M_TEXT_ANCHOR_VALUE.

You can add one of the following values to the above-mentioned values to determine whether an inquire type is supported for the string model currently being inquired.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For inquiring if an inquire type is supported																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																
M_HAS_DEFAULT		Inquires whether the specified inquire type has a default value.															
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type does not have a default value.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type has a default value.	c		e	f	g	a		i		j	k	l	m	n	b
M_SUPPORTED		Inquires whether the specified inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b
	UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
	M_FALSE	Specifies that the inquire type is not supported.	c		e	f	g	a		i		j	k	l	m	n	b
	M_TRUE	Specifies that the inquire type is supported.	c		e	f	g	a		i		j	k	l	m	n	b

Combination values for the values listed in For inquiring about a global setting of a string model; (exceptions might exist) and for the following value: M_STRING_SIZE.

You can add one of the following values to the above-mentioned values to cast the requested information to the required data type.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For casting information to the required data type.																	
InquireType		Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
	UserVarPtr - Possible values returned																

M_TYPE_AIL_DOUBLE	Casts the requested information to an *AIL_DOUBLE*.															
UserVarPtr extra info	Data type info: Data type: *address of an AIL_DOUBLE*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT	Casts the requested information to an *AIL_INT*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT32	Casts the requested information to an *AIL_INT32*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT32*	c		e	f	g	a		i		j	k	l	m	n	b
M_TYPE_AIL_INT64	Casts the requested information to an *AIL_INT64*.	c		e	f	g	a		i		j	k	l	m	n	b
UserVarPtr extra info	Data type info: Data type: *address of an AIL_INT64*	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the requested information, cast to an *AIL_INT64*. If the requested information does not fit into an *AIL_INT64*, this function will return **M_NULL** or truncate the information.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrPreprocess

Synopsis

Preprocess a Deep Learning OCR context.

Syntax

```
void MdlocrPreprocess(
    AIL_ID ContextDlocrId, //in
    AIL_INT64 ControlFlag  //in
)
```

Description

This function prepares a Deep Learning OCR context for a read or fine-tuning operation.

For a read context, this allows Deep Learning OCR to make internal refinements so it can execute optimized and robust character recognition processes. You must call this function before the first call to MdlocrRead().

For a fine-tuning context, this allows Deep Learning OCR to prepare the context for training. You must call this function before the first call to MdlocrFinetune().

Changes to the context or to any of its content often require you to preprocess the context again. To inquire a context's preprocessing state, call MdlocrInquire() with M_PREPROCESSED.

Saving a context does not save preprocessing changes. Upon restoration, you must preprocess the context again.

Parameters

ContextDlocrId

Specifies the identifier of the Deep Learning OCR context to preprocess. The context must have been previously allocated on the system using MdlocrAlloc().

ControlFlag

Specifies whether to preprocess the Deep Learning OCR context. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying whether to preprocess the context																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT	Preprocesses the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESET	Un-preprocesses the context. Un-preprocessing the context can be useful if you want to conserve system memory within an application and preserve context settings.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrRead

Synopsis

Read strings from a target image.

Syntax

```
void MdlocrRead(
    AIL_ID ContextDlocrId,   //in
    AIL_ID TargetImageBufId, //in
    AIL_ID ResultDlocrId,    //out
    AIL_INT64 ControlFlag    //in
)
```

Description

This function uses a Deep Learning OCR read context to read one or more strings from a target image. Results are stored in a Deep Learning OCR read result buffer. To get the results, use MdlocrGetResult().

By default, **MdlocrRead**() reads all text in an image within the default character height range. You can control global read settings using MdlocrControl(). You only need to add a string model if you want to restrict the strings that are found and read. MdlocrDefineModelFromResult() allows you to define the string model so that attributes are automatically set from a resulting string found using **MdlocrRead**(). If you only want to restrict the number of characters in the string or define a string model using an anchor, you can use MdlocrDefineModel(). Regardless of how you add a model, you can always constrain the characters that appear at certain positions using MdlocrControlConstraint() and adjust general string model settings using MdlocrControlStringModel().

Requirements for reading strings depend on the settings of the context and if used, string model-specific positional constraints and settings. To control these settings, use MdlocrControl(), MdlocrControlConstraint(), and MdlocrControlStringModel(). **MdlocrRead**() takes into account all settings.

Before performing a read operation, the Deep Learning OCR context must be in a preprocessed state (MdlocrPreprocess()).

To read a string, it must not only meet the context's requirements, but it must also follow the basic rules of a string. It must, for example, be a linear sequence of characters.

Strings are read randomly in an image. A string's position is the position of the center of the bounding box of its first character.

Before preprocessing, you must manually set M_TARGET_MAX_SIZE_X and M_TARGET_MAX_SIZE_Y, generally to the size of the target image. You can limit the read operation to a region of the target image buffer, using a rectangular ROI set with MbufSetRegion(). To increase performance, set M_TARGET_MAX_SIZE_X and M_TARGET_MAX_SIZE_Y to the size of the ROI.

If wanted strings have a known prefix or suffix, set M_TEXT_ANCHOR_MODE to M_TEXT_PREFIX or M_TEXT_SUFFIX respectively within your string model.

If the target image is not calibrated, results are calculated in the pixel coordinate system (pixel units). If the target image is calibrated, results are calculated in the world coordinate system (real-world units), but you can retrieve them in either world or pixel units. To specify this, call MdlocrControl() with M_RESULT_OUTPUT_UNITS.

In the presence of distortion, some results are meaningless when converted from real-world to pixel units (such as angle and scale). For example, if a string appears warped in the target image, but the camera calibration context compensates for this, the resulting angle is meaningful in the real-world coordinate system, and

meaningless in the pixel coordinate system. If complex distortions exist, correct the image before using it with Deep Learning OCR.

The read operation can occasionally take an unexpectedly long time to calculate. Use MdlocrControl() with M_TIMEOUT to set a maximum read time.

To improve the performance of your Deep Learning OCR read context, you can add valid read results to a Deep Learning OCR dataset, using MdlocrAddImageToDataset(). You can use this dataset to fine-tune a Deep Learning OCR read context on images from your particular application, using MdlocrFinetune().

Parameters

ContextDlocrId

Specifies the identifier of the Deep Learning OCR context to use for the read operation. The context must have been previously allocated on the required system using MdlocrAlloc(), and preprocessed using MdlocrPreprocess().

TargetImageBufId

Specifies the identifier of the target image buffer which contains the strings to read. The target image buffer must be a 1-band, 8-bit unsigned buffer. Its maximum size is 65536x65536 pixels, and its expeced maximum size should be set using MdlocrControl() with M_TARGET_MAX_SIZE_X and M_TARGET_MAX_SIZE_Y.

This image buffer can only have a rectangular region of interest (ROI), defined using MbufSetRegion() (with or without rotation). In addition, the region must be defined in vector format from a 2D graphics list (M_VECTOR or M_VECTOR_AND_RASTER). An error will be generated if the ROI is only in raster format (M_RASTER) or a non-rectangular shape.

ResultDlocrId

Specifies the identifier of the Deep Learning OCR result buffer in which to write the results of the read operation. The Deep Learning OCR result buffer must have been previously allocated on the required system using MdlocrAllocResult().

ControlFlag

Specifies the string reading mode.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the string matching mode																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_MODEL_BASED	Specifies that strings are read using the string models added to the Deep Learning OCR context. Only strings that meet the string model settings and positional constraints are read.	c		e	f	g	a		i		j	k	l	m	n	b
M_READ_ALL	Specifies to read all strings regardless of the string models added to the Deep Learning OCR context.	c		e	f	g	a		i		j	k	l	m	n	b

Compilation information

Header	Include AIL.h.

Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrRestore

Synopsis

Restore a Deep Learning OCR read or fine-tuning context from disk.

Syntax

```
AIL_ID MdlocrRestore(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID SysId,                //in
    AIL_INT64 ControlFlag,       //in
    AIL_ID *ContextDlocrIdPtr    //out
)
```

Description

This function restores a Deep Learning OCR read or fine-tuning context that was previously saved to a file, using MdlocrSave() or MdlocrStream(). This function restores all of the context's settings (including string model-specific positional constraints and settings) that were in effect when the context was saved. A restored context is not preprocessed; therefore, you must call MdlocrPreprocess() before performing a read using MdlocrRead().

When the restored Deep Learning OCR context is no longer required, release it using MdlocrFree() unless M_UNIQUE_ID was specified during allocation; if M_UNIQUE_ID was specified, the smart identifier manages the restored Deep Learning OCR context's lifetime and you must not manually free it.

Parameters

FileName

Specifies the name and path of the file from which to restore the Deep Learning OCR read or fine-tuning context. The function handles (internally) the opening and closing of the file. For easier use with other Aurora Imaging Library software products, files for Deep Learning OCR contexts should use the MDLOCR file extension.

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") [1]	Specifies the drive, directory, and name of the Deep Learning OCR context file (for example, "*C:\mydirectory\MyContext.mdlocr*"). Typically, Deep Learning OCR context files have an MDLOCR extension. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdlocr*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

SysId

Specifies the identifier of the system on which to restore the Deep Learning OCR read or fine-tuning context.

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the system identifier																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextDlocrIdPtr

Specifies the address of the variable in which to write the Deep Learning OCR context identifier or specifies the data type that the function should use to return the Deep Learning OCR context identifier.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For retrieving the identifier or specifying how to return it																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_NULL	Specifies that you will use this function's return value to obtain the identifier of the allocated Deep Learning OCR context ; in this case, a standard Aurora Imaging Library identifier of type *AIL_ID* is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_UNIQUE_ID	Specifies that you will use this function's return value to obtain the identifier of the allocated Deep Learning OCR context ; in this case, an Aurora Imaging Library smart identifier of type *AIL_UNIQUE_DLOCR_ID* is returned instead of a standard Aurora Imaging Library identifier. > This setting is only available when using C++11 (or later). An Aurora Imaging Library smart identifier manages the lifespan of the Aurora Imaging Library object it owns (similar to a *std::unique_ptr*). Note, you can use an Aurora Imaging Library smart identifier as though it were a standard Aurora Imaging Library identifier, except that you cannot use it to manually free the Deep Learning OCR context (it is freed automatically). For more information, see the Aurora Imaging Library smart identifiers subsection of the Aurora Imaging Library custom data types, void pointers, extensions, and portability functions section of Chapter 2: Key information and building an application.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the fine-tuning context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Deep Learning OCR fine-tuning context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b
Address in which to write the read context identifier	Specifies the address of an *AIL_ID* in which to write the identifier of the allocated Deep Learning OCR read context. If restoration fails, **M_NULL** is written as the identifier.	c		e	f	g	a		i		j	k	l	m	n	b

Return value

The returned value is the Deep Learning OCR context identifier either as a standard identifier (*AIL_ID*) or a smart identifier (*AIL_UNIQUE_DLOCR_ID*). If allocation fails, **M_NULL** is returned (or *nullptr* if M_UNIQUE_ID was specified).

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.

DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrSave

Synopsis

Save a Deep Learning OCR read or fine-tuning context to a file.

Syntax

```
void MdlocrSave(
    AIL_CONST_TEXT_PTR FileName, //in
    AIL_ID ContextDlocrId,       //in
    AIL_INT64 ControlFlag        //in
)
```

Description

This function saves all the information about the previously allocated Deep Learning OCR read or fine-tuning context to disk, including all of its global settings and string model-specific positional constraints and settings. This information can be reloaded, using MdlocrRestore() or MdlocrStream().

When you save the context, the preprocessing changes are not saved. Upon restoration, you must preprocess the context again.

Parameters

FileName

Specifies the name and path of the file in which to save the Deep Learning OCR read or fine-tuning context. The function internally handles the opening and closing of this file. For easier use with other Aurora Imaging Library software products, files for Deep Learning OCR read or fine-tuning contexts should use the MDLOCR file extension. If you specify a file that already exists, it will be overwritten.

Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the file name and path																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the file.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the Deep Learning OCR read or fine-tuning context file (for example, "*C:\mydirectory\MyContext.mdlocr*"). Typically, Deep Learning OCR context files have an MDLOCR extension. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdlocr*").	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

ContextDlocrId

Specifies the identifier of the Deep Learning OCR read or fine-tuning context to save.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.

MdlocrStream

Synopsis

Load, restore, or save a Deep Learning OCR read or fine-tuning context from/to a file or a memory stream.

Syntax

```
void MdlocrStream(
    AIL_TEXT_PTR MemPtrOrFileName, //in-out
    AIL_ID SysId,                  //in
    AIL_INT64 Operation,           //in
    AIL_INT64 StreamType,          //in
    AIL_DOUBLE Version,            //in
    AIL_INT64 ControlFlag,         //in
    AIL_ID *ContextDlocrIdPtr,     //in-out
    AIL_INT *SizeByteVarPtr        //out
)
```

Description

This function can load, restore, or save a Deep Learning OCR context from/to a file or memory stream. All information about the previously allocated context is saved, including all of the string model-specific settings and positional constraints. Preprocessing changes are not saved.

To inquire the number of bytes necessary to save a Deep Learning OCR read or fine-tuning context to memory stream, first call **MdlocrStream**() with M_INQUIRE_SIZE_BYTE.

The content saved to memory stream is equivalent to the content saved to file. In addition, any file saved using this function is equivalent to a file saved with MdlocrSave().

You can use this and other Aurora Imaging Library stream functions, for example, to save all required Aurora Imaging Library objects, as well as any other custom data, for your application to a memory stream. Once in a memory stream, you can write the stream to a single file or transfer it over a network. You are responsible for concatenating the streams and for saving the stream to file.

All of the context settings that were in effect when the context was saved will be restored. A loaded or restored context is not preprocessed. You must call MdlocrPreprocess() before performing a read with MdlocrRead().

Using **MdlocrStream**(), you can choose to save a backwards-compatible version of the Deep Learning OCR context, which will work using a version of Aurora Imaging Library that is up to one major release older than the current version (depending on which version is specified). However, all settings and features unique to the higher version will be ignored when restored using the lower version. Besides saving backwards-compatible versions, you can also load or restore Deep Learning OCR contexts saved using any Aurora Imaging Library version with which this function was available. Settings that do not exist in the lower version will be filled with default values when the Deep Learning OCR context is loaded or restored.

Parameters

MemPtrOrFileName
Accepts the address of one of the following (see the Parameter associations section for specifics on which is expected):

- AIL_TEXT_PTR [optionally, in C++: a reference to an AIL_STRING]
- Optionally, in C++: a reference to a std::vector of type AIL_UINT8

Specifies the file or memory stream.

See the Parameter associations section for possible values that can be returned/specified.

SysId

Specifies the identifier of the system on which to restore the Deep Learning OCR read or fine-tuning context.

See the Parameter associations section for possible values that can be specified.

Operation

Specifies the operation to perform on the Deep Learning OCR read or fine-tuning context.

See the Parameter associations section for possible values that can be specified.

StreamType

Specifies the type of stream in which to store/from which to restore the Deep Learning OCR read or fine-tuning context. Set this parameter to one of the values below:

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For specifying the type of stream																	
Value	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
M_FILE	Specifies a file stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_MEMORY	Specifies a memory stream. You are responsible for allocating a block of memory for the stream.	c		e	f	g	a		i		j	k	l	m	n	b

Version

Specifies the Aurora Imaging Library version of the Deep Learning OCR read or fine-tuning context.

See the Parameter associations section for possible values that can be specified.

ControlFlag

Reserved for future expansion and must be set to **M_DEFAULT**.

ContextDlocrIdPtr

Specifies the address of the variable in which to write or from which to read the identifier of the Deep Learning OCR context.

See the Parameter associations section for possible values that can be returned/specified.

SizeByteVarPtr

Specifies the address of the variable in which to write the size of the Deep Learning OCR context, in bytes. If size is not required, set this parameter to **M_NULL**.

> The size of the context varies depending on the Aurora Imaging Library version specified.

Parameter associations

The table below lists possible values for the **MemPtrOrFileName**, **SysId**, **Operation**, **Version**, and **ContextDlocrIdPtr** parameters.

- For performing the stream operation.

Board legend: c – clarity uhd, d – concord poe with toe, e – gentl, f – geviq, g – gige vision, a – host system, h – indio, i – iris gtx, o – Rapixo CXP Pro Octo, j – radient ev-cl, k – rapixo cl, l – rapixo cof, m – rapixo cxp, n – usb3 vision, b – video4linux2

For performing the stream operation.																	
Operation	Description	(c)	(d)	(e)	(f)	(g)	(a)	(h)	(i)	(o)	(j)	(k)	(l)	(m)	(n)	(b)
MemPtrOrFileName - Possible values passed or returned																
SysId																
Version																
ContextDlocrIdPtr - Possible values passed or returned																
M_INQUIRE_SIZE_BYTE	Inquires the number of bytes required to save a Deep Learning OCR context to memory stream. This operation is not supported when the StreamType parameter is set to M_FILE.															
MemPtrOrFileName	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the Deep Learning OCR context must be compatible when saved. The size of a Deep Learning OCR context might differ from one Aurora Imaging Library version to another.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDlocrIdPtr	Specifies the address of the variable containing the identifier of the Deep Learning OCR context.	c		e	f	g	a		i		j	k	l	m	n	b
M_LOAD	Loads the content of a specified file or memory stream into a previously allocated Deep Learning OCR context.	c		e	f	g	a		i		j	k	l	m	n	b

MemPtrOrFileName	Specifies the file or memory stream from which to load the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the context file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the context file (for example, "*C:\mydirectory\MyContext.mdlocr*"), when the StreamType parameter is set to M_FILE. Deep Learning OCR contexts typically have an MDLOCR file extension. The function handles (internally) the opening and closing of the file. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdlocr*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDlocrIdPtr	Specifies the address of the variable containing the identifier of the Deep Learning OCR context. All previous settings are overwritten with those of the loaded context, or are set to the default value if the settings didn't exist in the version of Aurora Imaging Library from which the context is being loaded.	c		e	f	g	a		i		j	k	l	m	n	b
M_RESTORE	Restores a Deep Learning OCR context from a file or memory stream and assigns it an Aurora Imaging Library identifier.	c		e	f	g	a		i		j	k	l	m	n	b

MemPtrOrFileName	Specifies the file or memory stream from which to restore the context.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the context file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the context file (for example, "*C:\mydirectory\MyContext.mdlocr*"), when the StreamType parameter is set to M_FILE. Deep Learning OCR contexts typically have an MDLOCR file extension. The function handles (internally) the opening and closing of the file. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdlocr*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which the object is located, and the block of memory must contain the entire object. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	Specifies the system on which to allocate the Deep Learning OCR context.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT_HOST	Specifies the default Host system of the current Aurora Imaging Library application.	c		e	f	g	a		i		j	k	l	m	n	b
System identifier	Specifies a valid system identifier, previously allocated using MsysAlloc().	c		e	f	g	a		i		j	k	l	m	n	b
Version	This parameter must be set to **M_DEFAULT**.	c		e	f	g	a		i		j	k	l	m	n	b
ContextDlocrIdPtr	Specifies the address of the variable in which to write the identifier of the Deep Learning OCR context. If the operation is not successful, **M_NULL** is returned.	c		e	f	g	a		i		j	k	l	m	n	b

M_SAVE	Saves a Deep Learning OCR context to a specified file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
MemPtrOrFileName	Specifies the file or memory stream.	c		e	f	g	a		i		j	k	l	m	n	b
M_INTERACTIVE	[This is only applicable to Windows] Opens a dialog box from which you can interactively specify the drive, directory, and name of the context file, when the StreamType parameter is set to M_FILE.	c		e	f	g	a		i		j	k	l	m	n	b
AIL_TEXT("*FileName*") 1	Specifies the drive, directory, and name of the context file (for example, "*C:\mydirectory\MyContext.mdlocr*"), when the StreamType parameter is set to M_FILE. Deep Learning OCR contexts typically have an MDLOCR file extension. The function handles (internally) the opening and closing of the file. If the file already exists, it will be overwritten. To specify a context file on a remote computer (under Distributed Aurora Imaging Library), prefix the specified file name string with "*remote:///*" (for example, "*remote:///C:\mydirectory\MyContext.mdlocr*").	c		e	f	g	a		i		j	k	l	m	n	b
MemPtr	Specifies the address of the block of memory, when the StreamType parameter is set to M_MEMORY. The block of memory should be of type *AIL_UINT8*. The specified address must correspond to the first memory address in which to write, and the block of memory must be large enough to stream the entire object. To determine the required size, call this function with M_INQUIRE_SIZE_BYTE. Note that when using a C compiler (not a C++ or other compiler), you must cast the *AIL_UINT8* pointer to an *AIL_TEXT_PTR*.	c		e	f	g	a		i		j	k	l	m	n	b
SysId	This parameter must be set to **M_NULL**.	c		e	f	g	a		i		j	k	l	m	n	b
Version	Specifies the Aurora Imaging Library version with which the Deep Learning OCR context must be compatible when saved.	c		e	f	g	a		i		j	k	l	m	n	b
M_DEFAULT	Specifies the current version of Aurora Imaging Library.	c		e	f	g	a		i		j	k	l	m	n	b
M_PROC_VERSION_100_SP7	Specifies the version as being Aurora Imaging Library 10.0 Service Pack 7.	c		e	f	g	a		i		j	k	l	m	n	b

		M_PROC_VERSION_110	Specifies the version as being Aurora Imaging Library 11.0.	c		e	f	g	a		i		j	k	l	m	n	b
	ContextDlocrIdPtr		Specifies the address of the variable containing the identifier of the Deep Learning OCR context to save.	c		e	f	g	a		i		j	k	l	m	n	b

[1] If you are passing the value in a variable, don't enclose it in AIL_TEXT().

Compilation information

Header	Include AIL.h.
Library	Use ail11.lib; aildlocr11.lib.
DLL	Requires ail11.dll; aildlocr11.dll.